

Ingegneria del Software Q&A

Federico Santi e Davide Rapp

14 luglio 2025

Modulo 1

1.1 - Come viene implementata l'ereditarietà multipla?

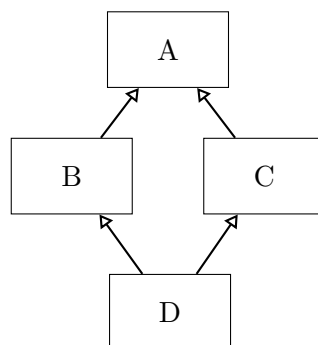
L'ereditarietà multipla si verifica quando, in una gerarchia di classi, almeno una classe deriva da due o più superclassi.

Ereditarietà multipla nelle lingue di programmazione

In linguaggi come **C#** e **Java**, l'ereditarietà multipla delle classi non è consentita, in quanto può generare numerose ambiguità nella risoluzione dei metodi e rendere la gerarchia difficile da mantenere. Al contrario, in **C++** l'ereditarietà multipla è supportata e utilizzata, ma richiede una gestione attenta per evitare conflitti.

Il problema del diamante

Un noto esempio di ambiguità legata all'ereditarietà multipla è il **problema del diamante**. Esso prende il nome dalla forma che assume la gerarchia di classi:



Esempio:

- La classe A definisce un metodo `doSomething()`.
- Le classi B e C ereditano da A e ridefiniscono il metodo.
- La classe D eredita sia da B che da C.

L'ambiguità nasce perché non è chiaro quale versione di `doSomething()` debba essere invocata da D.

Le interfacce come alternativa

C# e Java permettono invece l'ereditarietà multipla *solo delle interfacce*, che specificano la firma dei metodi, ma non il comportamento. Questo elimina l'ambiguità:

```
1 interface Flyable {
2     void fly();
3 }
4
5 interface Swimmable {
6     void swim();
7 }
8
9 class Seaplane implements Flyable, Swimmable {
10     public void fly() {
11         System.out.println("I'm flying");
12     }
13
14     public void swim() {
15         System.out.println("I'm swimming");
16     }
17 }
```

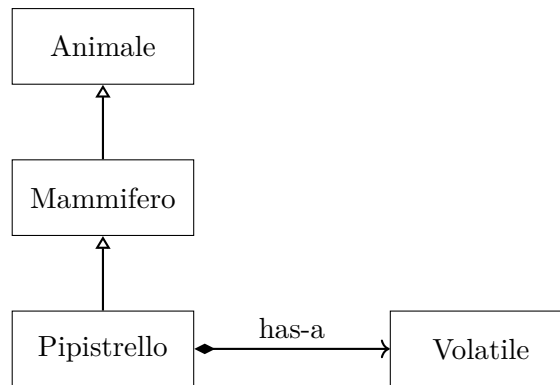
Listing 1: Ereditarietà multipla tramite interfacce in Java

Composizione e delega

Una tecnica alternativa all'ereditarietà multipla è l'uso della **composizione** e della **delega**. Si eredita da una sola superclasse significativa e si includono gli altri comportamenti tramite istanze delle altre *superclassi*, che prendono il nome di *ruoli*.

Esempio: Il Pipistrello Un pipistrello è sia un *mammifero* che un *volatile*. Poiché non è possibile ereditare da entrambe le classi, si adotta la composizione:

- Animale è la superclasse.
- Mammifero eredita da Animale.
- Volatile è una classe separata che possiede il metodo `vola()`.
- Pipistrello eredita da Mammifero ed è legato tramite composizione a Volatile (ne possiede un'istanza).



```

1 class Animale {}
2
3 class Mammifero extends Animale {}
4
5 class Volatile {
6     void vola() {
7         System.out.println("Volo!");
8     }
9 }
10
11 class Pipistrello extends Mammifero {
12     private Volatile volatilePart = new Volatile();
13
14     void vola() {
15         volatilePart.vola(); // delega
16     }
17 }

```

Listing 2: Composizione e delega

Combinazione: composizione-delega & interfacce

Un ulteriore approccio per simulare l'ereditarietà multipla in linguaggi che non la supportano (come Java o C#) è combinare l'uso delle **interfacce** con la **composizione e la delega**.

Si eredita da una sola classe base (es. **Mammifero**), mentre gli altri comportamenti (es. **volare**) vengono introdotti tramite interfacce e delegati a classi ausiliarie.

Esempio: il Pipistrello Un **Pipistrello** è un **Mammifero**, ma è anche un **Volatile**. In Java, non potendo ereditare da due classi, si procede così:

- **Animale** è la superclasse comune
- **Mammifero** eredita da **Animale**
- Si definisce l'interfaccia **Volatile** con il metodo **vola()**
- La classe **VolatileImpl** implementa il comportamento di volo

- Pipistrello eredita da Mammifero e implementa Volatile, delegando l'esecuzione ad un'istanza interna di VolatileImpl

Questo consente:

- Di mantenere l'ereditarietà da una sola classe concreta.
- Di simulare l'ereditarietà multipla in modo sicuro e modulare.
- Di separare responsabilità e favorire il riuso.

```

1 class Animale {}
2
3 class Mammifero extends Animale {}
4
5 interface Volatile {
6     void vola();
7 }
8
9 class VolatileImpl implements Volatile {
10     public void vola() {
11         System.out.println("Volo!");
12     }
13 }
14
15 class Pipistrello extends Mammifero implements Volatile {
16     private Volatile comportamentoVolo = new VolatileImpl();
17
18     public void vola() {
19         comportamentoVolo.vola(); // delega il comportamento
20     }
21 }

```

Listing 3: Simulazione di ereditarietà multipla con interfaccia e delega

1.2 - Si esegua una classificazione del polimorfismo secondo Cardelli-Wegner e si mostri l'implementazione del polimorfismo per inclusione.

Il **polimorfismo** è la capacità di un elemento di presentarsi in forme diverse in contesti differenti, o di elementi diversi di apparire sotto la stessa forma in uno specifico contesto.

Secondo la classificazione proposta da **Cardelli e Wegner**, il polimorfismo si suddivide in due grandi categorie:

- **Polimorfismo universale:** l'elemento assume un numero *potenzialmente infinito* di forme.
- **Polimorfismo ad hoc:** l'elemento assume un *numero finito* di forme.

1. Polimorfismo Universale

Per inclusione (inclusion polymorphism): usato nella programmazione orientata agli oggetti.

Consente a un oggetto di essere trattato come un'istanza della sua superclasse.

Le due tecniche coadiuvanti sono:

- **Overriding:** ridefinizione di un metodo della superclasse in una sottoclasse. È più sicuro quando il metodo è dichiarato astratto nella superclasse.
- **Binding dinamico:** avviene a tempo di esecuzione. Viene implementato tramite la **Virtual Method Table (VMT)**, una struttura dati associata a ciascuna classe che contiene i puntatori ai suoi metodi virtuali.

```
1  class Animale {
2      public virtual void verso() {
3          System.out.println("Verso");
4      }
5  }
6
7  class Cane extends Animale {
8      @Override
9      public void verso() {
10         System.out.println("Bau");
11     }
12 }
13
14 public class Main {
15     public static void main(String[] args) {
16         Animale a = new Cane(); // tipo statico: Animale
17                               // tipo dinamico: Cane
18         a.verso(); // Output: "Bau" grazie alla VMT
19     }
20 }
```

Listing 4: Overriding e VMT in C# (simulato)

Parametrico (parametric polymorphism): utilizzato nella programmazione generica. Una classe o funzione è definita con parametri di tipo che verranno specificati in fase di compilazione, generando istanze indipendenti, senza relazioni di ereditarietà.

```
1      class Box<T> {
2          private T contenuto;
3
4          public void set(T valore) {
5              contenuto = valore;
6          }
7
8          public T get() {
9              return contenuto;
10         }
11     }
12
13     public class Main {
14         public static void main(String[] args) {
15             Box<String> scatolaStringa = new Box<>();
16             scatolaStringa.set("Ciao");
17             System.out.println(scatolaStringa.get()); // Output: "Ciao"
18
19             Box<Integer> scatolaIntero = new Box<>();
20             scatolaIntero.set(42);
21             System.out.println(scatolaIntero.get()); // Output: 42
22         }
23     }
```

Listing 5: Esempio di generics - polimorfismo parametrico

2. Polimorfismo ad hoc

Overloading (sovraccarico): consente di definire metodi o operatori con lo stesso nome ma firme diverse. È risolto in fase di compilazione.

```
1      public class Calcolatrice {
2          public int somma(int a, int b) {
3              return a + b;
4          }
5
6          public double somma(double a, double b) {
7              return a + b;
8          }
9      }
```

Listing 6: Esempio di overloading

Coercion (coercizione): conversione implicita di tipi. Ad esempio, un intero può essere automaticamente convertito in un decimale se richiesto dal contesto.

```
1      int intero = 5;
2      double numero = intero; // coercizione implicita da int a
      double
```

Listing 7: Esempio di coercion (conversione implicita)

Esempio di polimorfismo per inclusione

Il seguente esempio mostra come il polimorfismo per inclusione venga implementato tramite `override` e `binding dinamico`:

```
1 public class A {
2     public virtual void Fun1(int x) {
3         Console.WriteLine("A-Fun1");
4     }
5
6     public virtual void Fun2(int y) {
7         Console.WriteLine("A-Fun2");
8     }
9 }
10
11 public class B : A {
12     public override void Fun1(int x) {
13         Console.WriteLine("Fun1_Ridefinita");
14     }
15
16     public virtual void Fun3(int z) {
17         Console.WriteLine("B-Fun3");
18     }
19 }
```

Listing 8: Esempio di polimorfismo per inclusione in C#

Analisi

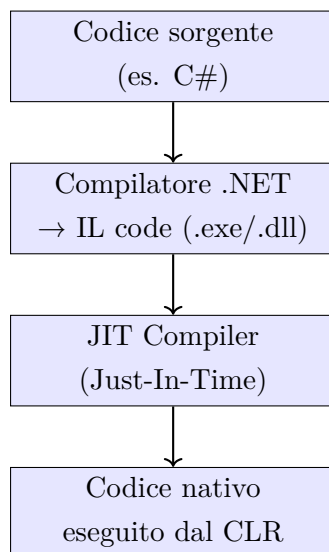
- La classe A definisce due metodi virtuali: `Fun1` e `Fun2`.
- La classe B estende A e **ridefinisce** il metodo `Fun1` usando `override`.
- A tempo di esecuzione, il metodo effettivamente invocato dipende dal tipo dinamico dell'oggetto, grazie al **binding dinamico** abilitato dalla **Virtual Method Table**.

Domanda 1.3 - Procedimento di compilazione ed esecuzione del codice .NET tramite CLR

Il **Common Language Runtime (CLR)** è l'ambiente di esecuzione del framework .NET, responsabile della gestione della memoria, della sicurezza, della compilazione JIT e dell'esecuzione del codice gestito.

Fasi di compilazione ed esecuzione

Il processo avviene in più fasi distinte:



- **Codice sorgente:** rappresenta il codice scritto dallo sviluppatore in un linguaggio .NET (C#, F# o VB.NET).
- **Compilatore .NET e IL:** il compilatore converte il codice sorgente in *Intermediate Language* (IL), indipendente dalla macchina fisica. Il risultato è un **CLI Assembly** (file .exe o .dll) che contiene anche un **manifest**, con:
 - il nome, la versione e i requisiti di sicurezza;
 - i metadati dei tipi;
 - le dipendenze esterne.
- **JIT Compiler:** il compilatore *Just-In-Time* traduce il codice IL in codice nativo specifico per la piattaforma al momento dell'esecuzione. Questo consente ottimizzazioni runtime.
- **Codice nativo eseguito dal CLR:** è il codice finale che viene eseguito sulla macchina host, controllato dal CLR che gestisce memoria, sicurezza, threading, eccezioni e garbage collection.

Gestione della memoria: CLR vs COM

Il **Garbage Collector** (GC) del CLR è responsabile della gestione automatica del ciclo di vita degli oggetti. Quando un oggetto non è più referenziato, viene deallocato automaticamente. Questo approccio differisce in modo sostanziale da quello adottato nel modello **COM** (**Component Object Model**).

COM: reference counting e distruzione deterministica Nel modello COM, la gestione della memoria si basa sul **reference counting**, ovvero ogni oggetto mantiene un contatore interno che traccia quanti riferimenti attivi lo puntano. Quando il contatore scende a zero, l'oggetto viene distrutto immediatamente. Questo garantisce una **distruzione deterministica** della memoria.

Tuttavia, il reference counting è vulnerabile a situazioni di **riferimenti circolari**, in cui due (o più) oggetti si referenziano reciprocamente. In tali casi, i contatori non scendono mai a zero, impedendo la distruzione automatica degli oggetti e generando **memory leak**.

CLR: garbage collection e riferimenti circolari Nel CLR, invece, non viene usato il reference counting. Il garbage collector scansiona periodicamente l'heap alla ricerca di oggetti non più raggiungibili da alcuna radice del programma. Questo consente al CLR di gestire anche strutture con **riferimenti circolari** in modo sicuro.

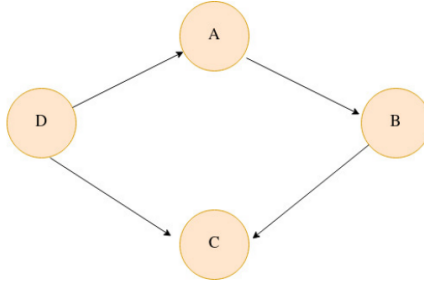


Figura 2: Riferimenti circolari

Figura 1: Esempio di riferimento circolare: gli oggetti si referenziano reciprocamente

Nella figura si osserva che ogni oggetto punta ad un altro formando un ciclo. In un sistema a reference counting (come COM), questo ciclo impedisce la distruzione degli oggetti anche se nessuno li raggiunge più. In .NET, invece, il GC riconosce che l'intero ciclo è inaccessibile dall'esterno e lo dealloca completamente.

Nota: il garbage collector in .NET opera in modo *non deterministico*, ovvero non è possibile sapere esattamente quando un oggetto verrà distrutto.

Input/Output su file

Il CLR supporta l'I/O tramite il namespace `System.IO`, che include classi per:

- leggere e scrivere file (`FileStream`, `StreamReader`, `StreamWriter`);
- accedere a directory e percorsi di file;
- gestire flussi binari e testuali.

Gestione delle eccezioni

Il CLR gestisce le eccezioni in modo strutturato: ogni eccezione è un oggetto derivato da `System.Exception`. I costrutti principali per la gestione sono:

- `throw` – per lanciare un'eccezione;
- `try/catch` – per intercettare e gestire l'errore;
- `finally` – per eseguire codice di pulizia, indipendentemente dal verificarsi dell'eccezione.

1.4 - Differenza tra tipi valore e tipi riferimento in .NET

Nel framework .NET, i tipi sono classificati dal **Common Type System (CTS)**, che fornisce un modello condiviso tra linguaggi orientati agli oggetti, funzionali e procedurali. Tutti i tipi derivano da `System.Object`, ma si dividono in:

Tipi Valore

- Contengono direttamente il dato;
- Sono allocati sullo **stack** o inline in altri oggetti;
- Non possono essere null (a meno che non siano `Nullable<T>`);
- Ogni variabile contiene una copia indipendente;
- Non supportano ereditarietà.

Esempi: `int`, `double`, `bool`, `char`, `struct`, `enum`.

Tipi Riferimento

- Contengono un riferimento all'oggetto allocato nell'**heap** gestito dal Garbage Collector;
- Due variabili possono puntare allo stesso oggetto;
- Possono essere null;
- Supportano ereditarietà, polimorfismo e incapsulamento.

Esempi: `class`, `string`, `array`, `delegate`, `interface`.

Boxing e Unboxing

Boxing è la conversione implicita di un tipo valore in tipo riferimento. **Unboxing** è l'operazione inversa (esplicita).

```
1 double d = 333.3;
2 Object obj = d;           // boxing
3 double d2 = (double)obj;  // unboxing
```

Passaggio dei parametri

In .NET, il comportamento del passaggio dei parametri dipende sia dalla *categoria* del tipo (valore o riferimento), sia dalla *modalità* di passaggio (`in`, `ref`, `out`).

Passaggio per valore (in) - modalità predefinita

- **Tipo valore:** viene passata una copia del contenuto → le modifiche non hanno effetto all'esterno del metodo;
- **Tipo riferimento:** viene passata una copia del riferimento → le modifiche all'*oggetto* hanno effetto, ma non la sostituzione del riferimento.

```
1 PostoRistorante posto = new PostoRistorante(1,1);
2 assegnaPosto(posto);
3 Console.WriteLine(posto); // (3,5) se classe, (1,1) se struct
4
5 static void assegnaPosto(PostoRistorante posto){
6     posto.NumeroTavolo = 3;
7     posto.NumeroPosto = 5;
8 }
```

Passaggio per riferimento (ref)

- L'argomento deve essere **già inizializzato**;
- Le modifiche influenzano direttamente il chiamante;
- Si può modificare l'intero riferimento, anche nel caso di tipi riferimento.

```
1 PostoRistorante posto = new PostoRistorante(1,1);
2 assegnaPosto(ref posto);
3 Console.WriteLine(posto); // (3,5) sia se struct che classe
4
5 static void assegnaPosto(ref PostoRistorante posto){
6     posto.NumeroTavolo = 3;
7     posto.NumeroPosto = 5;
8 }
```

Passaggio per riferimento (out)

- L'argomento **non deve essere inizializzato**;
- È obbligatorio inizializzarlo all'interno del metodo chiamato;
- Le modifiche influenzano direttamente il chiamante.

```
1 PostoRistorante posto;
2 assegnaPosto(out posto);
3
4 static void assegnaPosto(out PostoRistorante posto){
5     // posto.NumeroTavolo = 3; // Errore se prima non inizializzato
6     posto = new PostoRistorante(3,5); // Corretto
7 }
```

Riepilogo - Tabella di confronto

Caratteristica	Tipo Valore	Tipo Riferimento
Memoria	Stack o inline	Heap gestito (GC)
Contenuto	Dato diretto	Puntatore a oggetto
Nullabilità	Non null (default)	Può essere null
Prestazioni	Più veloce (no GC)	Più flessibile, ma con overhead
Ereditarietà	Non supportata	Supportata
Esempi	int, double, struct	class, string, array
Boxing/Unboxing	Necessario per usarli come oggetti	Non necessario
Passaggio per valore	Copia del dato	Copia del riferimento
Passaggio con ref/out	Diretta modifica al valore originale	Diretta modifica all'oggetto o al riferimento

1.5 - Garbage Collector in C#

Il **Garbage Collector (GC)** in C# è un componente fondamentale del **Common Language Runtime (CLR)**, responsabile della gestione automatica della memoria **heap**. Il suo obiettivo è liberare la memoria occupata da oggetti non più utilizzati, prevenendo errori comuni e migliorando la stabilità del programma.

Perché è necessario un Garbage Collector

Il GC risolve problematiche comuni della gestione manuale della memoria:

- **Dangling pointer:** puntatori a memoria deallocata, che possono causare comportamenti imprevedibili.
- **Doppia deallocazione:** tentativi ripetuti di liberare la stessa area di memoria.
- **Memory leak:** mancata deallocazione di oggetti non più utilizzati, che continuano a occupare memoria.

Svantaggi del GC

- Maggiore uso di risorse computazionali.
- Rilascio della memoria **non deterministico** (non controllabile esplicitamente).
- Incertezza sul momento e ordine di esecuzione della garbage collection.

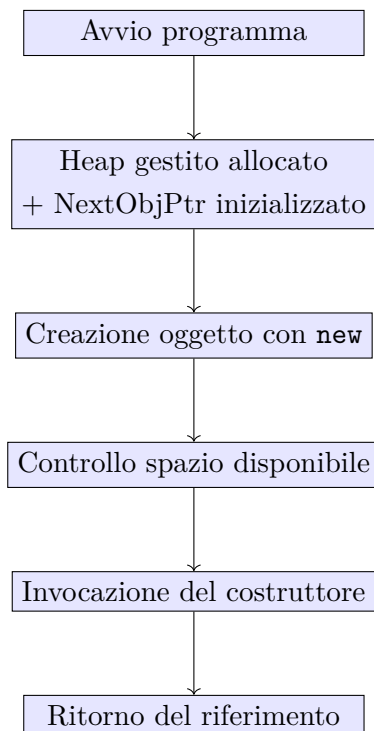
Funzionamento generale: approccio *tracing*

Il Garbage Collector adotta una strategia di tipo **tracing**, ovvero identifica tutti gli oggetti che sono ancora *raggiungibili* a partire da un insieme noto di *radici* (roots), come:

- variabili globali e campi statici;
- variabili locali nei frame attivi degli stack;
- registri della CPU che puntano a oggetti in heap.

Tutti gli oggetti non raggiungibili vengono marcati come *garbage* e saranno deallocati.

Il comportamento del GC è strettamente legato al modo in cui viene effettuata l'**allocazione della memoria** per gli oggetti. Lo schema seguente illustra i passaggi fondamentali del processo.



Spiegazione dei passaggi:

1. **Avvio programma:** il CLR riserva una porzione di memoria detta *managed heap*, usata per contenere gli oggetti creati dinamicamente.
2. **Heap gestito e NextObjPtr:** viene inizializzato un puntatore speciale, `NextObjPtr`, che indica il primo indirizzo libero dove collocare il prossimo oggetto.
3. **Creazione con new:** quando si istanzia un oggetto, il CLR calcola lo spazio necessario, che comprende:
 - i byte per i dati del tipo;
 - un *puntatore alla Virtual Method Table (VMT)* per supportare il dispatch dinamico dei metodi;
 - un campo `SyncBlockIndex`, utile per funzionalità avanzate come la sincronizzazione (lock).
4. **Controllo spazio disponibile:** se l'heap non ha spazio sufficiente, viene invocato il Garbage Collector. Se nemmeno il GC libera abbastanza memoria, viene lanciata un'eccezione `OutOfMemoryException`.
5. **Invocazione del costruttore:** una volta riservato lo spazio, viene chiamato il costruttore dell'oggetto.
6. **Ritorno del riferimento:** infine, viene restituito un riferimento all'oggetto allocato, che potrà essere usato dal programma.

Raccolta degli oggetti non utilizzati

Il GC parte da un insieme di **radici (roots)**:

- variabili globali/statiche;
- variabili locali nei frame attivi degli stack;
- registri CPU che contengono indirizzi validi.

Tutti gli oggetti *raggiungibili* da queste radici sono marcati come **vivi**. Gli altri sono considerati **garbage** e verranno eliminati.

Compattazione del managed heap

Dopo la rimozione degli oggetti non raggiungibili:

- la memoria del heap può risultare frammentata;
- il GC esegue una **compattazione**, spostando gli oggetti vivi per creare uno spazio contiguo;
- i riferimenti agli oggetti spostati vengono aggiornati;
- il `NextObjPtr` viene aggiornato.

Ruolo dei metadati

Il GC può determinare con precisione:

- quali campi di un oggetto contengono riferimenti ad altri oggetti;
- se un oggetto può essere considerato autonomamente garbage.

Gestione delle risorse `unmanaged`

Il GC gestisce solo memoria *gestita*. Per le risorse **`unmanaged`** (file, socket, connessioni DB...) è responsabilità del programmatore usare:

- metodi `Dispose()`;
- costrutti `using`;
- finalizzatori `~ClassName()` con cautela, poiché ritardano il rilascio.

1.6 - Passaggio dei parametri in C#

In C# esistono tre modalità di passaggio degli argomenti ai metodi: **in**, **ref**, **out**. Il comportamento cambia in base al tipo di dato (valore o riferimento) e alla parola chiave usata.

1. in (default) – passaggio per valore

- **Tipo valore:** viene passata una copia del contenuto → *le modifiche non hanno effetto all'esterno del metodo.*
- **Tipo riferimento:** viene passata una copia del riferimento → *le modifiche all'oggetto hanno effetto, ma non la sostituzione del riferimento.*

```
1 PostoRistorante posto = new PostoRistorante(1,1);
2 AssegnaPosto(posto);
3 Console.WriteLine(posto); // Se classe: (3,5) - Se struct: (1,1)
4
5 static void AssegnaPosto(PostoRistorante posto){
6     posto.NumeroTavolo = 3;
7     posto.NumeroPosto = 5;
8 }
```

2. ref – passaggio per riferimento (già inizializzato)

- **Tipo valore:** viene passato un riferimento al contenuto → *le modifiche influenzano direttamente il chiamante.*
- **Tipo riferimento:** viene passato un riferimento al riferimento → *è possibile sostituire l'oggetto originale.*

```
1 PostoRistorante posto = new PostoRistorante(1,1);
2 AssegnaPosto(ref posto);
3 Console.WriteLine(posto); // (3,5) sia classe che struct
4
5 static void AssegnaPosto(ref PostoRistorante posto){
6     posto.NumeroTavolo = 3;
7     posto.NumeroPosto = 5;
8 }
```

3. out – passaggio per riferimento (non inizializzato)

- Obbliga il metodo chiamato a inizializzare la variabile.
- È possibile restituire più valori da un metodo.

```
1 PostoRistorante posto;  
2 AssegnaPosto(out posto);  
3 Console.WriteLine(posto); // (3,5)  
4  
5 static void AssegnaPosto(out PostoRistorante posto){  
6     // posto.NumeroTavolo = 3; // Errore: non inizializzato  
7     posto = new PostoRistorante(3,5); // Obbligatorio  
8 }
```

1.7 - Concetto di delegato in C#

In **C#**, un **delegato** è un tipo che rappresenta un riferimento a un metodo con una firma specifica. È simile a un puntatore a funzione in C/C++, ma è *type-safe* e compatibile con la programmazione orientata agli oggetti.

Definizione

Un delegato incapsula:

- la **firma** del metodo (parametri e tipo di ritorno),
- un **metodo statico** oppure un metodo di **istanza**,
- un elenco (chiamato *invocation list*) di metodi da invocare.

```
1 // Definizione del delegato
2 delegate void Operazione(string messaggio);
3
4 // Metodo compatibile
5 static void StampaConsole(string messaggio) {
6     Console.WriteLine(messaggio);
7 }
8
9 // Uso del delegato
10 Operazione op = StampaConsole;
11 op("Ciao dal delegato!"); //OUTPUT: Ciao dal delegato!
```

Scenari d'uso principali

- **Elaborazione asincrona:** permettono a un metodo di accettare un delegato da invocare al termine di un'elaborazione lunga.
- **Elaborazione cooperativa:** un metodo esterno fornisce una parte dell'algoritmo, ad es. il confronto tra elementi in un MergeSort.
- **Gestione degli eventi:** i delegati sono alla base del modello di eventi in C#, in cui uno o più listener si registrano per essere notificati.

Multicast delegate (invocation list)

Un delegato può puntare a più metodi. Tutti vengono invocati **sincronamente**, in ordine.

```
1 void Saluto1(string nome) => Console.WriteLine("Ciao " + nome);
2 void Saluto2(string nome) => Console.WriteLine("Benvenuto " + nome);
3
4 Operazione saluti = Saluto1;
5 saluti += Saluto2; // Aggiunge al multicast
6
7 saluti("Anna"); // Invoca entrambi in ordine
```

Parametri by-ref e valori di ritorno

- Se si usano **ref** nei parametri, ogni metodo nell'invocation list riceve e può modificare lo stesso riferimento → le modifiche sono visibili ai successivi.
- Se i metodi restituiscono un valore, **verrà mantenuto solo il valore di ritorno dell'ultimo metodo** della lista.

Delegati anonimi e lambda expression

Oltre alla definizione classica, è possibile creare delegati anche in modo anonimo:

```
1 // Lambda expression
2 Operazione op = msg => Console.WriteLine("Messaggio: " + msg);
3 op("Delegato lambda");
```

Esempio di elaborazione (callback) asincrona

```
1 class Impiegato {
2     public delegate void Report(string fase);
3     public Report attivita;
4
5     public void Lavora() {
6         attivita?.Invoke("Inizio");
7         // attivita lavorativa...
8         attivita?.Invoke("Fine");
9     }
10 }
11
12 class Capo {
13     public void Notifica(string msg) {
14         Console.WriteLine("Capo riceve: " + msg);
15     }
16 }
17
18 // Uso
19 Capo c = new Capo();
20 Impiegato i = new Impiegato();
21 i.attivita = c.Notifica;
22 i.Lavora();
```

Esempio di elaborazione cooperativa

```
1 delegate bool Confronto(int a, int b);
2
3 void Ordina(int[] array, Confronto criterio) {
4     for (int i = 0; i < array.Length - 1; i++) {
5         for (int j = i + 1; j < array.Length; j++) {
6             if (criterio(array[j], array[i])) {
7                 int temp = array[i];
8                 array[i] = array[j];
9                 array[j] = temp;
10            }
11        }
12    }
13 }
14
15 int[] numeri = { 5, 2, 8, 1 };
16 Ordina(numeri, (a, b) => a < b); // Ordina in modo crescente
17 // Output: 1 2 5 8
```

1.8 - Concetto di evento in C#

Gli **eventi** in C# sono un meccanismo che permette di notificare a uno o più oggetti che qualcosa è accaduto. Sono costruiti sopra i **delegati**, di cui rappresentano una *protesi sicura e controllata*.

Differenza tra delegate e event

Un delegato, se dichiarato come campo pubblico, permette a chiunque di:

- accedere direttamente all'elenco dei metodi registrati;
- sovrascriverlo arbitrariamente (es. con `= null`).

Questo è pericoloso: rompe l'incapsulamento. Gli **eventi**, invece:

- incapsulano il delegato;
- espongono solo le operazioni sicure: `+=` per iscriversi e `-=` per annullare l'iscrizione;
- impediscono l'invocazione esterna.

Modello a oggetti: sender, handler, subscriber

- **Event sender**: l'oggetto che genera (scatena) l'evento.
- **Event receiver / subscriber**: l'oggetto che desidera essere notificato.
- **Event handler**: il metodo chiamato quando l'evento si verifica.

Dichiarazione e utilizzo

```
1 public class Motore
2 {
3     public event EventHandler Accensione;
4
5     public void Avvia()
6     {
7         Console.WriteLine("Motore in avvio...");
8         Accensione?.Invoke(this, EventArgs.Empty); // Notifica
9     }
10 }
11
12 public class Cruscotto
13 {
14     public void Reagisci(object sender, EventArgs e)
15     {
16         Console.WriteLine("Il motore   acceso!");
17     }
18 }
19
20 class Program
```

```

21 {
22     static void Main()
23     {
24         Motore m = new Motore();
25         Cruscotto c = new Cruscotto();
26
27         m.Accensione += c.Reagisci; // Iscrizione all'evento
28
29         m.Avvia(); // Output: Motore in avvio... /           Il motore
                       acceso!
30     }
31 }

```

Listing 9: Definizione di evento e notifica

Delegati per eventi: `EventHandler` e `EventHandler<T>`

In C#, un evento è basato su un delegato. Il delegato standard utilizzato è:

```

1 public delegate void EventHandler(object sender, EventArgs e);

```

Questo delegato rappresenta un metodo che riceve:

- **sender**: l'oggetto che ha generato l'evento;
- **e**: i dati associati all'evento, incapsulati in una classe derivata da `EventArgs`.

Se l'evento non deve comunicare informazioni specifiche, si può usare direttamente `EventArgs.Empty`. Ma quando si vogliono passare dati personalizzati (es. una soglia superata, una percentuale, un messaggio, ecc.), è buona pratica creare una classe che eredita da `EventArgs`, ad esempio:

```

1 public class MioEventArgs : EventArgs
2 {
3     public int Valore { get; set; }
4 }

```

A questo punto, invece del delegato base, si usa la versione generica:

```

1 public event EventHandler<MioEventArgs> EventoPersonalizzato;

```

Questo approccio è più sicuro e leggibile, perché evita cast espliciti nel gestore e consente l'uso della tipizzazione forte:

```

1 void handler(object sender, MioEventArgs e)
2 {
3     Console.WriteLine(e.Valore); // Nessun cast necessario
4 }

```

Anche se `MioEventArgs` eredita comunque da `EventArgs`, l'uso del delegato generico `EventHandler<T>` permette al compilatore di verificare i tipi corretti a tempo di compilazione, migliorando la manutenzione e l'affidabilità del codice.

In sintesi:

- `EventHandler` è adatto per eventi generici o senza dati;
- `EventHandler<T>` è preferibile quando servono dati specifici, perché evita il cast e rende il codice più robusto.

Operazioni ammesse su eventi

- `evento += handler`: iscrizione di un gestore.
- `evento -= handler`: rimozione del gestore.
- `evento = ...` **non è permesso** all'esterno della classe.
- `evento()`: solo la classe che ha dichiarato l'evento può invocarlo.

Delegati multicast e comportamento

Gli eventi si basano su delegati, che possono essere **multicast**:

- Gli handler vengono invocati in ordine di registrazione.
- Se un handler ha parametri **ref**, le modifiche saranno visibili ai successivi.
- Il valore di ritorno dell'evento (se previsto) sarà quello dell'ultimo handler.

Quando usare gli eventi

Gli eventi sono utili per:

- **GUI** (es. click su un bottone).
- **Notifiche asincrone** (es. completamento di un task).
- **Osservatori** (pattern Observer / Publisher-Subscriber).

Riepilogo differenze delegate / event

Caratteristica	Delegate	Event
Visibilità	Campo accessibile ovunque	Solo <code>+=</code> / <code>-=</code> ammessi esternamente
Sicurezza	Poco sicuro (rischio sovrascrittura)	Sicuro: incapsulato
Utilizzo	Puntatore a funzione generico	Meccanismo formale per notifiche
Multicast	Sì	Sì (gestione automatica)
Esempio tipico	Callback, filtro, strategia	Click, notifiche, observer

1.9 - Metaprogrammazione in C#

La **metaprogrammazione** è la capacità di un programma di esaminare, modificare o generare codice a runtime, sfruttando informazioni su se stesso. In C#, questo avviene principalmente tramite la **riflessione** (`System.Reflection`).

Perché usare la metaprogrammazione

Sebbene non sempre necessaria, la metaprogrammazione è utile in scenari come:

- **Serializzazione dinamica** (es. JSON/XML);
- **ORM** (Object-Relational Mapping) come Entity Framework;
- **Test automatici** e mocking;
- **Plugin system** e **caricamento dinamico** di moduli;
- **Generazione di UI** o validazione automatica dei dati.

In Java questo corrisponde al pacchetto `java.lang.reflect`, ma in C# è molto più integrato e potente grazie ai metadati e alla possibilità di generare codice in IL (Intermediate Language).

Concetto di riflessione

La riflessione permette a un programma di:

- **Esplorare tipi** a runtime (nomi di classi, metodi, proprietà...);
- **Creare oggetti** anche senza conoscerne il tipo a compile-time;
- **Invocare metodi e accedere a campi** scoperti dinamicamente.

Tutti i tipi in .NET sono rappresentati da oggetti della classe `System.Type`.

```
1 object o = "ciao";  
2 Type tipo = o.GetType();  
3 Console.WriteLine(tipo.FullName); // System.String
```

Listing 10: Scoprire il tipo di un oggetto a runtime

Esempio pratico: invocazione dinamica di un metodo

```
1 Type t = typeof(string);  
2 MethodInfo metodo = t.GetMethod("ToUpper", Type.EmptyTypes);  
3 string risultato = (string)metodo.Invoke("testo", null);  
4 Console.WriteLine(risultato); // TESTO
```

Listing 11: Invocazione dinamica con Reflection

Creazione dinamica di oggetti

Activator.CreateInstance() consente di istanziare tipi dinamicamente:

```
1 Type t = typeof(List<string>);
2 object lista = Activator.CreateInstance(t);
3 ((List<string>)lista).Add("prova");
```

Listing 12: Creazione di oggetti senza usare new

Ispezione di assembly

È possibile caricare un assembly esterno ed esplorarne i tipi:

```
1 Assembly asm = Assembly.Load("NomeMioAssembly");
2 foreach (Type tipo in asm.GetTypes())
3     Console.WriteLine(tipo.Name);
```

Attributi personalizzati

Gli **attributi** sono metadati aggiuntivi che descrivono tipi, metodi, proprietà, ecc. Possono essere letti a runtime tramite riflessione.

```
1 [AttributeUsage(AttributeTargets.Method)]
2 public class AutoreAttribute : Attribute
3 {
4     public string Nome { get; }
5     public AutoreAttribute(string nome) { Nome = nome; }
6 }
7
8 class Libro {
9     [Autore("Dante")]
10    public void Leggi() { }
11 }
```

Listing 13: Creazione di un attributo personalizzato

Per leggerli:

```
1 MethodInfo m = typeof(Libro).GetMethod("Leggi");
2 object[] attr = m.GetCustomAttributes(typeof(AutoreAttribute), false);
3 Console.WriteLine(((AutoreAttribute)attr[0]).Nome); // Dante
```

Metaprogrammazione avanzata: Reflection.Emit

System.Reflection.Emit permette di generare dinamicamente:

- classi, metodi, proprietà;
- interi assembly in codice intermedio (IL).

Esempi:

- compilatori a runtime (es. LINQ Expressions, Roslyn);
- serializzatori ad alte prestazioni;
- interpreti di linguaggi.

Riepilogo

Tecnica	Scopo
Reflection	Ispezione e manipolazione dinamica di tipi/metodi/oggetti
Activator.CreateInstance	Istanza oggetti in modo dinamico
Custom Attributes	Annotazioni lette a runtime, usate per configurazioni e metadata
Reflection.Emit	Generazione dinamica di codice IL (assembly, classi, metodi)

1.10 - Principali difetti (bad design) nella progettazione software

Una buona progettazione rende il software:

- più **affidabile** (meno errori);
- più **efficiente** (esegue meglio);
- più **manutenibile** (facile da aggiornare);
- più **riusabile** e **economico**.

Esistono però alcune caratteristiche tipiche di una cattiva progettazione (**bad smells**) che compromettono questi vantaggi. Vediamole una per una con esempi:

1. Rigidità

Un sistema è **rigido** quando una piccola modifica si propaga in modo incontrollato nel resto del codice. Questo accade se i moduli sono troppo strettamente accoppiati.

Esempio:

```
1 // Tutte le classi dipendono da ReportExporter
2 class ReportManager {
3     public void Esporta() {
4         ReportExporter esporta = new ReportExporter();
5         esporta.toPDF();
6     }
7 }
```

Se si cambia `ReportExporter` per esportare in Excel, si devono modificare tutte le classi che la usano. Una soluzione corretta sarebbe usare un'interfaccia `IExporter`, e applicare il principio di *inversione delle dipendenze*.

2. Fragilità

Un sistema è **fragile** quando una modifica innocua introduce bug in parti non collegate direttamente.

Esempio:

```
1 public class Fattura {
2     public decimal CalcolaTotale() {
3         return CalcolaIVA() + TotaleSenzaIVA;
4     }
5
6     private decimal CalcolaIVA() {
7         return TotaleSenzaIVA * 0.22m;
8     }
9 }
```

Modificare il metodo `CalcolaIVA` per gestire più aliquote può causare errori nella logica di `CalcolaTotale`, anche se il metodo non è stato toccato. Questo è tipico di una progettazione poco modulare o con responsabilità sovrapposte.

3. Immobilità

Un modulo è **immobile** quando è difficile riutilizzarlo in altri contesti, anche se potenzialmente utile.

Esempio:

```
1 public class ValidaEmail {
2     public bool CheckEmail(string email) {
3         // Validazione, ma con dipendenze a Logger, DB ecc.
4         LogDB.Save(email);
5         return Regex.IsMatch(email, pattern);
6     }
7 }
```

Non si può riutilizzare `ValidaEmail` in un altro progetto senza portare con sé anche la logica di logging e accesso al database. Una soluzione è separare le responsabilità in più classi, rendendo il modulo indipendente.

4. Viscosità

La **viscosità** si manifesta quando è più semplice implementare una modifica con un **hack** (una scorciatoia non strutturata) che seguendo la buona progettazione.

Esistono due tipi:

- **Viscosità del design:** il codice corretto richiede troppo sforzo rispetto alla scorciatoia.
- **Viscosità dell'ambiente:** l'ambiente di sviluppo scoraggia il refactoring (compilazioni lente, test lenti...).

Esempio:

```
1 // Soluzione "veloce", ma errata: si copia e incolla codice
2 if (tipo == "cliente") {
3     Console.WriteLine("Cliente");
4 }
5 if (tipo == "fornitore") {
6     Console.WriteLine("Fornitore");
7 }
```

L'aggiunta di un nuovo tipo avviene duplicando codice invece di usare una struttura dati o un pattern. Se scrivere una soluzione elegante richiede troppo sforzo, è probabile che il design o l'ambiente siano "viscosi".

Cause comuni di cattiva progettazione

- Mancanza di competenze progettuali;
- Pressioni esterne (scadenze, budget);
- Codice scritto in fretta, senza refactoring;
- Evoluzione del progetto senza adeguata manutenzione del design iniziale.

1.11 - Principio di Singola Responsabilità (Single Responsibility Principle - SRP)

Il **principio di singola responsabilità**, formulato da Robert C. Martin (Uncle Bob), è uno dei principi SOLID della programmazione orientata agli oggetti. Esso afferma che:

“Una classe dovrebbe avere una, e una sola, ragione per cambiare.”

In altre parole, ogni modulo (classe, metodo, funzione o variabile) deve avere una sola responsabilità, cioè deve occuparsi di un solo aspetto del sistema. Se una classe ha più responsabilità, una modifica a una di esse può interferire con le altre, rendendo il codice:

- difficile da modificare in sicurezza;
- difficile da comprendere;
- difficile da riutilizzare o testare.

Responsabilità = Motivo per cambiare

Una buona guida per applicare il principio è domandarsi: *quali sono i motivi per cui questa classe potrebbe cambiare?* Se i motivi sono più di uno, probabilmente sta violando il SRP.

Esempio pratico: Macchina Fotografica

Supponiamo di voler rappresentare una macchina fotografica digitale.

```
1 class MacchinaFotografica {
2     public void ScattaFoto() {
3         // codice per scattare una foto
4     }
5
6     public void RegistraVideo() {
7         // codice per registrare un video
8     }
9 }
```

Questa classe ha due responsabilità distinte:

- Gestire la logica per scattare fotografie.
- Gestire la logica per registrare video.

Se domani cambia il modo in cui salviamo le foto (es. nuovo formato), dobbiamo modificare la classe, ma potremmo inavvertitamente introdurre bug anche nella logica dei video.

Applicazione del principio SRP

Separiamo le responsabilità in due classi distinte:

```

1 class FotoCamera {
2     public void ScattaFoto() {
3         // logica per le foto
4     }
5 }
6
7 class VideoCamera {
8     public void RegistraVideo() {
9         // logica per i video
10    }
11 }

```

Eventualmente possiamo anche creare una classe che le combina:

```

1 class MacchinaFotografica {
2     private FotoCamera foto;
3     private VideoCamera video;
4
5     public void ScattaFoto() => foto.ScattaFoto();
6     public void RegistraVideo() => video.RegistraVideo();
7 }

```

Ora ogni classe ha una singola responsabilità e può essere modificata, testata o riutilizzata indipendentemente.

Conclusione

Il SRP aiuta a mantenere il codice:

- più modulare;
- più leggibile;
- più testabile e riusabile;
- meno soggetto a bug dovuti a cambiamenti in aree non correlate.

È uno dei principi fondamentali per la progettazione di software robusto e manutenibile.

1.12 - Principio Aperto/Chiuso con almeno un esempio

Il **principio aperto/chiuso** afferma che:

“Il software dovrebbe essere aperto all'estensione, ma chiuso alla modifica.”

Questo significa che è possibile aggiungere nuove funzionalità senza modificare il codice esistente. Per ottenere questo risultato si utilizzano:

- **interfacce e classi astratte**, che fungono da astrazione comune;
- **ereditarietà e polimorfismo**, per creare nuove estensioni.

Si distinguono due forme di estensione:

- **Ereditarietà di interfaccia**: le classi derivate implementano metodi definiti in un'interfaccia o in una classe astratta, senza modificarla;
- **Ereditarietà dell'implementazione**: si estende una classe base riutilizzando codice comune già esistente.

Questo approccio:

- migliora la modularità del sistema;
- previene l'introduzione di bug nel codice esistente;
- favorisce la scalabilità.

Esempio pratico: Calcolo delle spese aziendali

Supponiamo di avere una classe `CalcolatoreSpeseAziendali` che calcola il costo del personale aziendale a partire da una collezione di dipendenti.

1. Si definisce una **interfaccia astratta** `Impiegato` con il metodo `getSalario()`.
2. Si implementano varie classi concrete:
 - `Manager`
 - `AmministratoreDelegato`
 - `Programmatore`
3. Tutte queste classi implementano `getSalario()` in base al proprio ruolo.
4. La classe `CalcolatoreSpeseAziendali` ha un metodo:

```
1 double CalcolaSpese(List<Impiegato> impiegati) {  
2     double totale = 0;  
3     for (Impiegato i : impiegati) {  
4         totale += i.getSalario();  
5     }  
6     return totale;  
7 }
```

5. Se un giorno si volesse aggiungere un nuovo tipo di dipendente, come **Venditore**, basterebbe:

- creare la nuova classe **Venditore** che implementa **Impiegato**;
- ridefinire il metodo `getSalario()`.

Senza modificare il codice della classe **CalcolatoreSpeseAziendali**.

Questo esempio dimostra come il principio OCP renda il sistema **estensibile** e **robusto**, evitando modifiche al codice già testato.

1.13 - Principio di Sostituibilità di Liskov con almeno un esempio

Il **Principio di Sostituibilità di Liskov** (Liskov Substitution Principle, LSP) definisce il rapporto tra una classe e le relative sottoclassi. Esistono due versioni di questo principio:

- **Versione debole:** a ogni riferimento della classe base (classe padre) deve poter corrispondere, senza violare la correttezza del programma, un'istanza della sottoclasse (Classe figlia).;
- **Versione forte:** ogni programma che utilizza istanze della classe base non percepisce variazioni logiche nell'utilizzo delle istanze delle relative sottoclassi.;

In pratica, significa che se una classe B estende una classe A, allora ovunque venga usata un'istanza di A, si deve poter usare anche un'istanza di B senza modificare il comportamento corretto del programma.

Il principio si fonda sul concetto di **design by contract**, secondo cui ogni metodo impone:

- **Precondizioni:** requisiti minimi che devono essere soddisfatti dal chiamante, affinché il metodo invocato possa essere eseguito correttamente (es. l'array passato non deve essere null, oppure un valore numerico deve essere maggiore di zero).
- **Postcondizioni:** requisiti garantiti dal metodo dopo l'esecuzione corretta (es. un metodo `Ordina()` deve restituire una sequenza ordinata; `CalcolaArea()` deve restituire un valore positivo).

Affinché una sottoclasse rispetti il principio, deve:

- non rafforzare le **precondizioni**;
- non indebolire le **postcondizioni**;
- mantenere la **semantica** dei metodi ereditati;
- non introdurre eccezioni o effetti collaterali inattesi.

Violazione del LSP - Esempio

```
1 class Uccello {
2     public virtual void Vola() {
3         Console.WriteLine("Sto volando!");
4     }
5 }
6
7 class Pinguino : Uccello {
8     public override void Vola() {
9         throw new NotSupportedException("I pinguini non volano!");
10    }
11 }
```

Listing 14: Violazione del LSP con la classe Pinguino

In questo esempio, la classe `Pinguino` estende `Uccello` ma viola il principio:

- Il metodo `Vola()` viene ridefinito con un comportamento incompatibile.
- Un client che usa un oggetto `Uccello` si aspetta sempre che `Vola()` sia invocabile.
- Se si passa un `Pinguino`, il programma solleverà un'eccezione imprevista.

Applicazione corretta del LSP - Esempio

```
1 abstract class Animale { }
2
3 abstract class Uccello : Animale {
4     public abstract void Vola();
5 }
6
7 class Piccione : Uccello {
8     public override void Vola() {
9         Console.WriteLine("Il piccione vola.");
10    }
11 }
12
13 class Pinguino : Animale {
14     public void Nuota() {
15         Console.WriteLine("Il pinguino nuota.");
16    }
17 }
```

Listing 15: Corretto rispetto del LSP separando le gerarchie

In questo secondo esempio:

- Solo gli uccelli volanti ereditano da `Uccello`.
- `Pinguino` eredita da `Animale`, evitando l'ereditarietà impropria.
- Nessun metodo viene ridefinito in modo incompatibile.

1.14 - Principio di segregazione delle interfacce con almeno un esempio

Il **principio di segregazione delle interfacce** afferma che:

“I client non devono essere costretti a dipendere da interfacce che non utilizzano.”

Questo significa che è preferibile definire **interfacce specifiche e più piccole**, piuttosto che una sola interfaccia molto generica e ampia (*fat interface*). In questo modo:

- ogni classe implementa solo i metodi realmente utili al proprio ruolo;
- si evitano dipendenze inutili;
- si migliora la manutenibilità e la modularità del codice.

Esempio: stampante multifunzione

Supponiamo di voler modellare una stampante che possa:

- stampare documenti;
- scansionare documenti;
- inviare fax.

Un errore comune è creare una sola interfaccia che accorpi tutte queste operazioni:

```
1 interface IMultifunzione {  
2     void Stampa();  
3     void Scansiona();  
4     void InviaFax();  
5 }
```

Ora, se volessimo creare una semplice **stampante termica**, saremmo costretti a implementare anche metodi che non ci servono:

```
1 class StampanteTermica : IMultifunzione {  
2     public void Stampa() { /* ok */ }  
3  
4     public void Scansiona() {  
5         throw new NotImplementedException(); // inutile!  
6     }  
7  
8     public void InviaFax() {  
9         throw new NotImplementedException(); // inutile!  
10    }  
11 }
```

Problema: la classe dipende da metodi che non usa. Questo la rende fragile e meno riusabile.

Soluzione con ISP

Applichiamo il principio di segregazione, creando interfacce più piccole e specializzate:

```
1 interface IStampante {  
2     void Stampa();  
3 }  
4  
5 interface IScanner {  
6     void Scansiona();  
7 }  
8  
9 interface IFax {  
10    void InviaFax();  
11 }
```

Ora ogni classe può implementare solo ciò che le serve:

```
1 class StampanteTermica : IStampante {  
2     public void Stampa() { ... }  
3 }  
4  
5 class StampanteMultifunzione : IStampante, IScanner, IFax {  
6     public void Stampa() { ... }  
7     public void Scansiona() { ... }  
8     public void InviaFax() { ... }  
9 }
```

Vantaggi:

- Ogni client dipende solo dai metodi rilevanti.
- Aggiungere o modificare funzionalità è più semplice.
- Il codice è più chiaro e meno soggetto a modifiche a cascata.

Nota storica

Il principio fu formalizzato anche dopo un caso reale: Xerox, sviluppando una stampante multi-funzione (fax, pinzatura, scansione), aveva progettato una sola classe con tutte le funzionalità. Ogni modifica diventava difficile e rischiosa. L'applicazione del principio ISP risolse il problema, separando le responsabilità in interfacce specifiche come `IScanFogli` e `IPinzaFogli`.

1.15 - Principio di inversione delle dipendenze con almeno un esempio

Il **principio di inversione delle dipendenze** afferma che:

“I moduli di alto livello (clienti) non devono dipendere dai moduli di basso livello (fornitori dei servizi). Entrambi devono dipendere da astrazioni.”

Il motivo di questa affermazione è che i moduli di basso livello contengono la logica implementativa e dettagli concreti. Se i moduli di alto livello dipendessero direttamente da essi, il sistema risulterebbe:

- **Rigido**: ogni modifica a un modulo concreto può propagarsi a molti altri moduli;
- **Fragile**: una modifica a un dettaglio può introdurre errori in altre parti del sistema;
- **Immobilizzabile**: il riuso del codice diventa difficile, poiché i moduli di alto livello sono strettamente legati a quelli di basso livello.

Il DIP risolve questi problemi introducendo una separazione tra:

- moduli astratti (**interface**, **abstract class**) — stabili e poco soggetti a cambiamento;
- moduli concreti — modificabili senza impattare i moduli client.

Vantaggi del DIP

- I cambiamenti non si propagano: **design for change**.
- I moduli sono più **riusabili**: **design for reuse**.

Esempio pratico

Supponiamo di avere il seguente codice, che viola il principio DIP:

```
1 class BigliettoNormale {
2     public void Stampa() { ... }
3 }
4
5 class GestoreVenditaBiglietti {
6     private BigliettoNormale biglietto;
7
8     public GestoreVenditaBiglietti() {
9         biglietto = new BigliettoNormale();
10    }
11
12    public void Vendi() {
13        biglietto.Stampa();
14    }
15 }
```

Problema: `GestoreVenditaBiglietti` dipende direttamente dalla classe concreta `BigliettoNormale`. Se vogliamo aggiungere un nuovo tipo di biglietto, ad esempio `BigliettoVIP`, dovremmo modificare il gestore.

Soluzione con DIP

Definiamo un'astrazione:

```
1 interface IBiglietto {  
2     void Stampa();  
3 }
```

Facciamo implementare l'interfaccia alle classi concrete:

```
1 class BigliettoNormale : IBiglietto {  
2     public void Stampa() { ... }  
3 }  
4  
5 class BigliettoVIP : IBiglietto {  
6     public void Stampa() { ... }  
7 }
```

Modifichiamo ora il gestore in modo che dipenda dall'interfaccia:

```
1 class GestoreVenditaBiglietti {  
2     private IBiglietto biglietto;  
3  
4     public GestoreVenditaBiglietti(IBiglietto biglietto) {  
5         this.biglietto = biglietto;  
6     }  
7  
8     public void Vendi() {  
9         biglietto.Stampa();  
10    }  
11 }
```

Vantaggio: se vogliamo introdurre una nuova classe `BigliettoVIP`, non serve modificare il gestore — basta passare una nuova istanza al costruttore:

```
1 IBiglietto vip = new BigliettoVIP();  
2 GestoreVenditaBiglietti g = new GestoreVenditaBiglietti(vip);  
3 g.Vendi();
```

Conclusione

Il DIP è fondamentale per:

- ridurre il coupling tra moduli;
- migliorare la manutenibilità;
- favorire estensibilità e testabilità del codice.

1.16 - Principi per l'architettura dei package

Per garantire un sistema modulare, manutenibile e riusabile, l'architettura dei package in un software deve seguire alcuni principi fondamentali, divisi in due gruppi: quelli che regolano l'organizzazione interna dei package e quelli che definiscono le loro relazioni.

1. Principi per l'organizzazione interna dei package

- **REP (Principio di equivalenza di riuso/rilascio)**

Le classi dovrebbero essere raggruppate in package solo se vengono riutilizzate insieme. Inoltre, un package dovrebbe poter essere rilasciato e versionato come unità autonoma. Se un package non è mantenuto attivamente, non dovrebbe essere usato da altri.

Esempio: Se ho una libreria per la gestione delle date e degli orari, non ha senso includere anche funzioni di logging nello stesso package, poiché non verrebbero riutilizzate insieme.

- **CCP (Principio di chiusura comune)**

Le classi che cambiano per le stesse ragioni dovrebbero essere raggruppate nello stesso package. Così facendo, ogni modifica tende a impattare un solo package, rendendo il sistema più manutenibile.

Esempio: Se diverse classi cambiano ogni volta che viene aggiornato il formato dei dati utente, allora dovrebbero stare tutte in un package come `UserDataFormat`.

- **CRP (Principio di riutilizzo comune)**

Le classi che non sono riutilizzate insieme non dovrebbero trovarsi nello stesso package. Questo evita che ogni piccolo cambiamento costringa a ricompilare e ridistribuire classi non correlate.

Esempio: Una classe `EmailValidator` non dovrebbe condividere il package con una `FileCompressor` se non vengono mai usate insieme.

Nota: I tre principi sono in tensione tra loro. CCP incoraggia package più grandi (classi che cambiano insieme), mentre REP e CRP spingono verso package piccoli e più specifici. Una buona prassi è:

- in fase di sviluppo: privilegiare il CCP per facilitare le modifiche;
- in fase di stabilizzazione: rifattorizzare per favorire REP e CRP e massimizzare il riuso.

2. Principi per la gestione delle dipendenze tra package

- **ADP (Acyclic Dependencies Principle)**

Il grafo delle dipendenze tra package deve essere aciclico. La presenza di cicli rende difficile modificare o compilare anche solo una parte del sistema.

Esempio: Se il package UI dipende da `BusinessLogic`, e quest'ultimo dipende da UI, si ha un ciclo. Per risolverlo, si può:

- introdurre un'interfaccia in un terzo package (`IBusinessLogic`);

- far dipendere UI da quell'interfaccia;
- fare in modo che `BusinessLogic` la implementi.

- **SDP (Stable Dependencies Principle)**

Un package dovrebbe dipendere solo da package più stabili di lui. Un package è stabile se è poco soggetto a modifiche (per esempio, contiene solo interfacce astratte o classi di base).

Esempio: È accettabile che un modulo `GestioneOrdini` dipenda da `ILogger`, che è stabile. Ma non il contrario, perché modificare `GestioneOrdini` implicherebbe cambiare anche `ILogger`, rompendo stabilità.

- **SAP (Stable Abstractions Principle)**

I package più stabili dovrebbero essere anche astratti, cioè composti da interfacce o classi base, in modo che possano essere facilmente estesi senza essere modificati.

Esempio: Un package contenente l'interfaccia `IDatabaseAdapter` dovrebbe essere astratto e stabile. Le implementazioni concrete (es. `MySQLAdapter`, `PostgreSQLAdapter`) saranno invece instabili e collocate in package separati.

3. Organizzazione a livelli

Un'architettura ben progettata dispone i package in livelli:

- **Livelli bassi:** package stabili e astratti, difficili da modificare, ma facili da estendere;
- **Livelli alti:** package instabili, contenenti implementazioni concrete che possono cambiare spesso.

Questo approccio segue il principio **Open/Closed**: i package stabili non vengono modificati, ma estesi.

1.17 - Pattern Singleton con esempi

Risposta: Il pattern **Singleton** prevede che una classe abbia una sola istanza; chiunque può accedervi attraverso l'unica istanza citata. Una classe che implementa il pattern Singleton è strutturata nel seguente modo:

```
1 public class Singleton {
2
3     private static Singleton instance = null; // riferimento all'istanza
4
5     private Singleton() {} // costruttore privato
6
7     public static Singleton getInstance() {
8         if (instance == null)
9             instance = new Singleton();
10        return instance;
11    }
12
13    public void metodo() {
14        // operazioni
15    }
16 }
```

Listing 16: Implementazione base di Singleton

Il costruttore è privato, in quanto assicura che venga creata al più un'unica istanza della classe **Singleton**. Il metodo statico `getInstance()` controlla se è già esistente un'istanza della classe: in caso affermativo restituisce l'istanza creata in precedenza, altrimenti ne crea una nuova.

Una classe con soli membri statici non rappresenta un'alternativa al pattern Singleton, in quanto non permette di creare istanze personalizzate in base al contesto; inoltre, non consente l'utilizzo di un numero arbitrario di interfacce.

Un esempio classico di utilizzo del pattern Singleton è un **logger**, che centralizza le stampe di log di un'applicazione:

```

1 public class Logger {
2
3     private static Logger instance;
4
5     private Logger() {}
6
7     public static Logger getInstance() {
8         if (instance == null)
9             instance = new Logger();
10        return instance;
11    }
12
13    public void log(String message) {
14        System.out.println("[LOG] " + message);
15    }
16 }
17
18 // Utilizzo
19 Logger.getInstance().log("Sistema avviato");

```

Listing 17: Esempio di Logger Singleton

Questo garantisce che tutte le operazioni di logging passino dallo stesso oggetto condiviso, evitando duplicazioni, conflitti o perdita di messaggi.

1.18 - Pattern Observer con esempi

Risposta: Il pattern **Observer** viene utilizzato nel caso in cui un oggetto (detto *subject*) debba notificare un cambiamento del proprio stato interno a uno o più oggetti (detti *observer*). Si definiscono le seguenti classi:

- **Subject:** dichiara i metodi `Attach()` (o `Register()`), `Detach()` (o `Unregister()`) e `Notify()`.
- **Observer:** dichiara il metodo `Update()`.

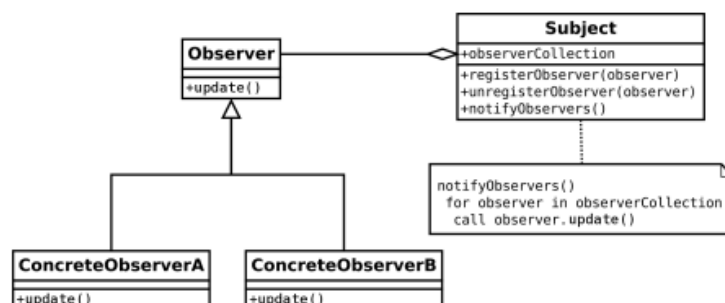
Per registrarsi presso **Subject**, un **Observer** invoca il metodo `Attach()`. Successivamente, in caso di evento, **Subject** chiama il metodo `Notify()`, che esegue `Update()` di tutti gli **Observer** che si sono registrati. Infine, un **Observer** si può disiscrivere da un **Subject**, sfruttando il metodo `Detach()`.

Il pattern **Observer** è una delle cinque possibili soluzioni nel caso in cui si voglia aggiornare elementi in base al cambiamento del subject. Altri possibili quattro approcci sono:

- **Class based:** ogni **Subject** ha un riferimento diretto all'**Observer** da notificare.
- **Interface based:** ogni **Subject** possiede un riferimento all'interfaccia implementata dagli **Observer**, aumentando il disaccoppiamento.
- **Delegate based:** ogni **Subject** contiene un delegato pubblico, a cui gli **Observer** si registrano.
- **Event based:** un **event** si occupa della registrazione e disiscrizione automatica degli **Observer**.

Un esempio di utilizzo del pattern **Observer** potrebbe essere l'iscrizione da parte di diversi dispositivi elettronici a un **SensoreTemperatura**; nel caso in cui la temperatura cambi, tutti i dispositivi registrati al subject **Sensore** vengono notificati.

Diagramma UML:



1.18.1 - Pattern Flyweight con esempi

Il pattern *Flyweight* consente la condivisione di oggetti leggeri tra più entità, in modo tale che il loro utilizzo non sia troppo costoso. L'oggetto condiviso (flyweight) viene utilizzato da più client in modo efficace nello stesso momento. In particolare:

- l'oggetto non deve essere distinguibile da un oggetto non condiviso;
- l'oggetto non deve fare ipotesi sul contesto in cui opera.

Una corretta condivisione dei flyweight tra i client viene garantita dall'utilizzo di una **FlyweightFactory**, che gestisce l'istanza e il riutilizzo dei flyweight.

Esistono due stati associati al pattern:

- **Stato intrinseco**: memorizzato nel flyweight, indipendente dal contesto e condivisibile tra client;
- **Stato estrinseco**: dipendente dal contesto, memorizzato dal client e passato al flyweight al momento dell'invocazione.

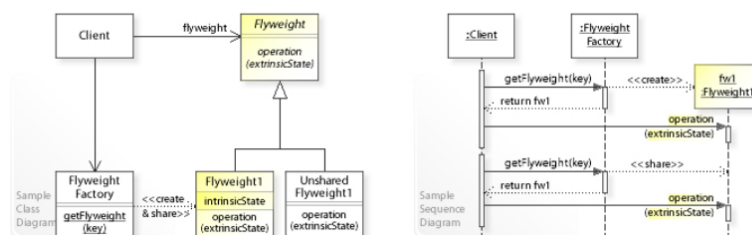
Esempio pratico: Gestione di alberi in una mappa geografica interattiva

Si supponga di voler rappresentare milioni di alberi all'interno di una mappa interattiva (ad esempio per un'applicazione GIS o un videogioco open world). Invece di creare un oggetto distinto per ogni albero (che causerebbe un grande utilizzo di memoria), si applica il pattern Flyweight:

- Lo stato **intrinseco** è rappresentato dalla specie dell'albero (ad esempio tipo di foglie, colore, altezza media, modello 3D, texture, etc.), che è comune a tutti gli alberi della stessa specie.
- Lo stato **estrinseco** è rappresentato dalla posizione di ciascun albero sulla mappa (coordinate X, Y, altezza sul livello del mare), che invece è specifico per ogni istanza logica e viene fornito dinamicamente dal client.

In questo modo si risparmia memoria condividendo un singolo oggetto per ogni specie di albero, evitando la duplicazione inutile delle stesse informazioni strutturali.

Diagramma UML:



1.19 - Pattern Strategy con esempi

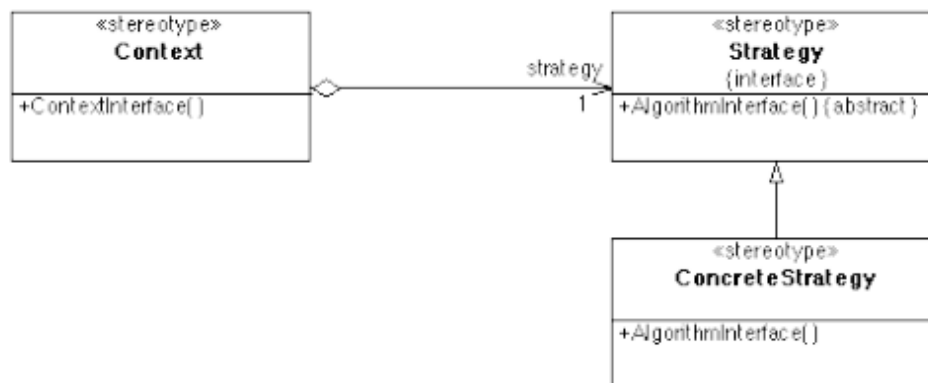
Il pattern **Strategy** è un pattern di tipo comportamentale e viene utilizzato quando si ha una famiglia di algoritmi, intercambiabili tra loro, che implementano una determinata funzionalità. A questo scopo si dichiara un'interfaccia dotata di metodi che svolgono la funzionalità richiesta; si dichiara inoltre, per ogni algoritmo, una classe concreta che implementa l'interfaccia e i relativi metodi definiti precedentemente.

Nel caso in cui un cliente, che dipende unicamente dall'interfaccia, voglia eseguire le relative funzionalità, non deve fare assunzioni su quale strategia sia stata adottata.

Un esempio di utilizzo del pattern **Strategy** è il seguente:

- Interfaccia **ISortStrategy** che espone il metodo **sort()**.
- Classe concreta **MergeSortStrategy** che implementa l'interfaccia **ISortStrategy** e realizza il metodo **sort()**, usando l'algoritmo di merge sort.
- Classe concreta **QuickSortStrategy** che implementa l'interfaccia **ISortStrategy** e realizza il metodo **sort()**, usando l'algoritmo di quick sort.
- Classe concreta **BubbleSortStrategy** che implementa l'interfaccia **ISortStrategy** e realizza il metodo **sort()**, usando l'algoritmo di bubble sort.
- Classe **Cliente** che ha un riferimento a un oggetto di tipo **ISortStrategy**.

Diagramma UML:



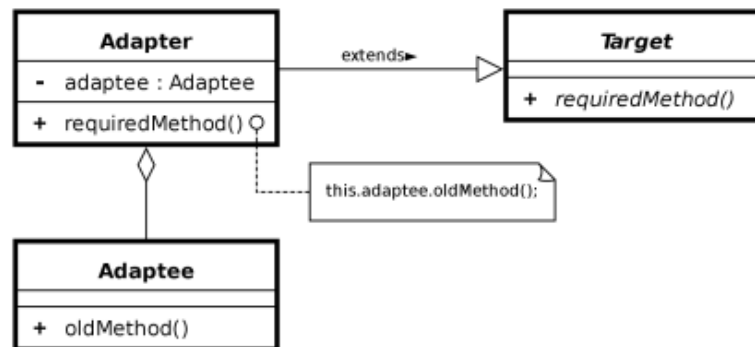
1.20 - Pattern Adapter con esempi

Il pattern **Adapter** è un pattern di tipo strutturale. Si supponga che un *Cliente* dipenda da un'interfaccia **Target** già definita, la quale espone un certo metodo. Esiste una certa classe **Adaptee** che realizza tale metodo, ma ha un'interfaccia incompatibile con l'interfaccia **Target**. Di conseguenza, per risolvere questo problema, si utilizza una classe **Adapter**, che implementa **Target** e richiama il metodo di **Adaptee**, mascherandolo come una chiamata al metodo di **Target**.

Un esempio di utilizzo del pattern **Adapter** è il seguente:

- Si ha un'interfaccia **IDisegnatore2D** che espone il metodo `disegnaFigura(Figura2D f)`.
- Si ha una classe **Cliente** che vuole disegnare una figura 2D usando l'interfaccia **IDisegnatore2D**.
- Si ha una classe **MotoreGrafico3D** fornita da una libreria esterna, che disegna oggetti 3D tramite il metodo `renderOggetto(Oggetto3D o)`, ma anche 2D passando il parametro `z=0`.
- Si realizza una classe **Adapter2Dto3D**, che implementa **IDisegnatore2D** e contiene un riferimento a un oggetto **MotoreGrafico3D**.
- Quando il client invoca il metodo `disegnaFigura()` sull'interfaccia **IDisegnatore2D**, l'**Adapter2Dto3D** converte la **Figura2D** in un **Oggetto3D** compatibile (stesse coordinate (x,y) e coordinata `z=0`) e chiama `renderOggetto()` di **MotoreGrafico3D**, senza che il client si accorga della differenza.

Diagramma UML:



Domanda 1.21 - Pattern Decorator con esempi

Il **Pattern Decorator** è un pattern strutturale che consente di aggiungere dinamicamente nuove responsabilità a un oggetto, senza alterarne la struttura di base. È un'alternativa all'ereditarietà quando si desidera evitare una gerarchia troppo estesa e rigida.

Vantaggi rispetto all'ereditarietà:

- Le funzionalità possono essere aggiunte a run-time (dinamicamente), non solo a compile-time.
- Il codice rimane più pulito, modulare e facilmente testabile.
- Viene rispettato il **principio di singola responsabilità**, poiché ogni decoratore aggiunge una singola funzione specifica.

Struttura generale:

- **Component**: interfaccia comune per gli oggetti a cui possono essere aggiunte responsabilità.
- **ConcreteComponent**: implementazione di base di **Component**.
- **Decorator**: classe astratta che implementa **Component** e contiene un riferimento ad un oggetto **Component**.
- **ConcreteDecorator**: aggiunge nuove funzionalità al **Component**.

Esempio pratico: personalizzazione del tè

Si vogliono gestire diverse varianti di tè, permettendo l'aggiunta dinamica di ingredienti come latte, miele o ghiaccio.

```
1 public interface ITea {  
2     String getDescription();  
3     double getCost();  
4 }
```

Listing 18: Interfaccia base

```
1 public class GreenTea implements ITea {  
2     public String getDescription() {  
3         return "Green Tea";  
4     }  
5  
6     public double getCost() {  
7         return 1.50;  
8     }  
9 }
```

Listing 19: ConcreteComponent

```

1 public abstract class TeaExtra implements ITea {
2     protected ITea baseTea;
3
4     public TeaExtra(ITea tea) {
5         this.baseTea = tea;
6     }
7 }

```

Listing 20: Decorator astratto

```

1 public class TeaWithMilk extends TeaExtra {
2     public TeaWithMilk(ITea tea) {
3         super(tea);
4     }
5
6     public String getDescription() {
7         return baseTea.getDescription() + ", with milk";
8     }
9
10    public double getCost() {
11        return baseTea.getCost() + 0.30;
12    }
13 }

```

Listing 21: ConcreteDecorator - Latte

```

1 public class TeaWithHoney extends TeaExtra {
2     public TeaWithHoney(ITea tea) {
3         super(tea);
4     }
5
6     public String getDescription() {
7         return baseTea.getDescription() + ", with honey";
8     }
9
10    public double getCost() {
11        return baseTea.getCost() + 0.50;
12    }
13 }

```

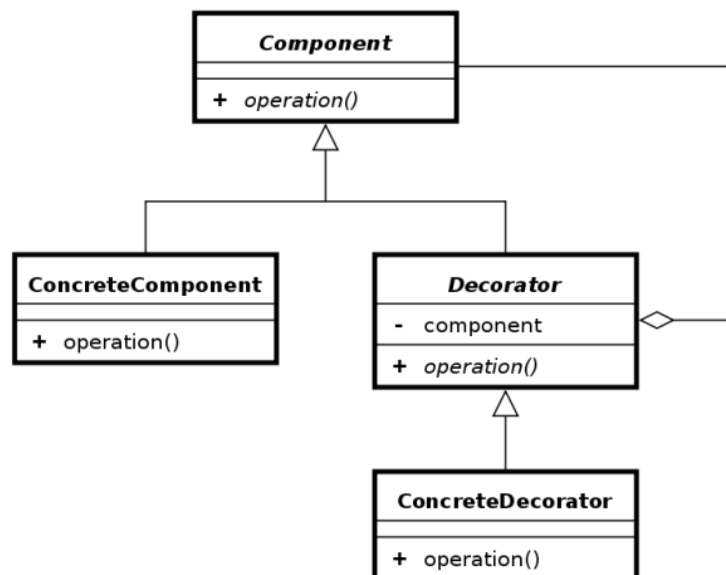
Listing 22: ConcreteDecorator - Miele

Utilizzo:

```
1 ITea myTea = new GreenTea();
2 myTea = new TeaWithMilk(myTea);
3 myTea = new TeaWithHoney(myTea);
4
5 System.out.println(myTea.getDescription()); // Green Tea, with milk,
        with honey
6 System.out.println(myTea.getCost());        // 2.30
```

Il pattern Decorator permette di combinare in modo flessibile funzionalità multiple, come l'aggiunta di ingredienti al tè, senza la necessità di creare sottoclassi per ogni combinazione possibile.

Diagramma UML:



1.22 - Pattern Composite con esempi

Il **Pattern Composite** è un pattern strutturale che permette di trattare in modo uniforme oggetti singoli (foglie) e oggetti composti (nodi) all'interno di strutture gerarchiche ad albero.

Strutture ad albero: come funzionano

Una struttura ad albero è composta da:

- un **nodo radice** (root) da cui parte la struttura;
- **nodi interni** che possono avere figli;
- **foglie**, cioè nodi terminali che non hanno figli.

Ogni nodo può contenere altri nodi (quindi è un oggetto che contiene altri oggetti dello stesso tipo), ma le foglie no.

Modellare un albero in Java o C#

La struttura ad albero si implementa creando una classe con un campo (ad esempio una lista) che contiene riferimenti ad altri oggetti dello stesso tipo:

```
1 class Nodo {
2     String nome;
3     List<Nodo> figli;
4
5     Nodo(String nome) {
6         this.nome = nome;
7         this.figli = new ArrayList<>();
8     }
9
10    void aggiungiFiglio(Nodo figlio) {
11        figli.add(figlio);
12    }
13
14    void stampa(String indentazione) {
15        System.out.println(indentazione + nome);
16        for (Nodo f : figli)
17            f.stampa(indentazione + "  ");
18    }
19 }
```

Listing 23: Struttura ad albero generica

Pattern Composite: struttura e ruoli

Il pattern Composite organizza il tutto nel seguente modo:

- **Component**: interfaccia o classe astratta base con i metodi comuni (es. `operazione()`, `add()`, `remove()`).
- **Leaf**: rappresenta un oggetto semplice, cioè una foglia senza figli.

- **Composite:** rappresenta un oggetto composto da altri oggetti (nodi interni), e mantiene una collezione di Component.
- **Client:** interagisce con gli oggetti Component (senza sapere se si tratti di foglie o composite).

Trasparenza vs Sicurezza

- **Trasparenza:** tutti i metodi (anche `add()` e `remove()`) sono in `Component`. Le foglie possono lanciare eccezioni se vengono invocati metodi non validi.
- **Sicurezza:** `add()` e `remove()` sono solo nei `Composite`. Il client deve distinguere tra `Leaf` e `Composite`.

Esempio pratico: Gerarchia aziendale

Supponiamo di modellare la gerarchia di un'azienda:

```

1 abstract class Impiegato {
2     String nome;
3     Impiegato(String nome) { this.nome = nome; }
4     abstract void mostraDettagli();
5 }

```

Listing 24: Component

```

1 class Programmatore extends Impiegato {
2     Programmatore(String nome) { super(nome); }
3
4     @Override
5     void mostraDettagli() {
6         System.out.println("Programmatore: " + nome);
7     }
8 }

```

Listing 25: Leaf

```

1 class Manager extends Impiegato {
2     List<Impiegato> sottoposti = new ArrayList<>();
3
4     Manager(String nome) { super(nome); }
5
6     void aggiungi(Impiegato i) { sottoposti.add(i); }
7
8     @Override
9     void mostraDettagli() {
10        System.out.println("Manager: " + nome);
11        for (Impiegato i : sottoposti)
12            i.mostraDettagli();
13    }
14 }

```

Listing 26: Composite

```

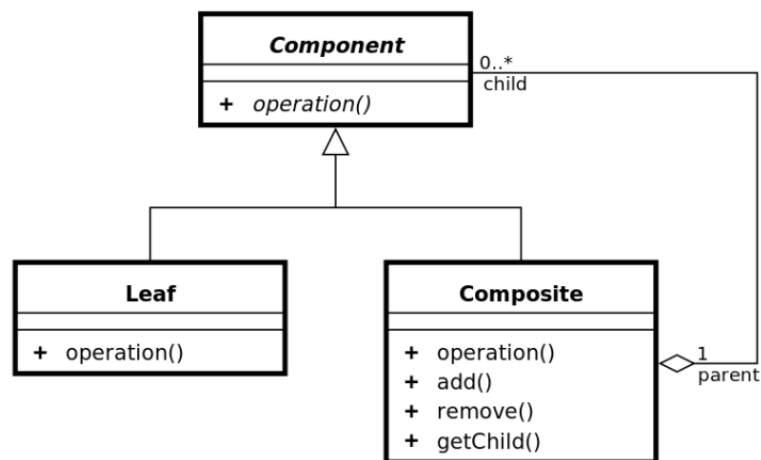
1 public class Azienda {
2     public static void main(String[] args) {
3         Manager direttore = new Manager("Anna");
4         Programmatore p1 = new Programmatore("Luca");
5         Programmatore p2 = new Programmatore("Giulia");
6         Manager teamLead = new Manager("Marco");
7
8         teamLead.aggiungi(p1);
9         teamLead.aggiungi(p2);
10        direttore.aggiungi(teamLead);
11
12        direttore.mostraDettagli();
13    }
14 }

```

Listing 27: Uso del Client

Questo esempio mostra chiaramente come un client possa trattare **Impiegato** (Component) senza preoccuparsi se si tratta di una foglia (**Programmatore**) o di un nodo composito (**Manager**).

Diagramma UML:



1.23 - Pattern Visitor con esempi

Il **Pattern Visitor** è un pattern *comportamentale* che permette di separare le **operazioni** dagli **oggetti** su cui vengono eseguite. In altre parole, consente di definire nuove funzionalità senza modificare la struttura delle classi su cui operano.

Perché è utile? Quando abbiamo una gerarchia di classi (es. `Libro`, `CD`, `DVD`, ecc.) e vogliamo eseguire più operazioni diverse su di esse (es. vendita, prestito, sconto), il pattern Visitor permette di:

- evitare di modificare continuamente le classi `Libro`, `CD`, ecc.;
- mantenere il principio **open/closed**: possiamo aggiungere nuove operazioni creando nuove classi `Visitor`, senza modificare la gerarchia;
- mantenere il principio di **singola responsabilità**: ogni operazione (es. vendita, prestito) sta in una classe dedicata.

Componenti principali del pattern:

- **Visitor**: interfaccia che dichiara un metodo per ogni tipo di oggetto da visitare, es. `visitLibro(Libro l)`, `visitCD(CD c)`;
- **ConcreteVisitor**: implementa `Visitor` e contiene il comportamento concreto (es. calcolo del prezzo, prestito, ecc.);
- **Element**: interfaccia o classe astratta comune a tutti gli oggetti che possono essere visitati. Definisce il metodo `accept(Visitor v)`;
- **ConcreteElement**: es. `Libro`, `CD`, implementano `Element` e nel metodo `accept` richiamano il visitatore su se stessi;
- **ObjectStructure**: collezione di oggetti (`List<Element>`) da visitare. Rappresenta la "struttura" da attraversare.

Meccanismo chiave: Double Dispatch

Il metodo `accept()` permette il **double dispatch**, ovvero:

1. l'oggetto sa chi lo sta visitando (`Visitor`);
2. il `Visitor` sa quale tipo concreto sta visitando (`Libro`, `CD`, ecc.).

Questo meccanismo consente di avere comportamenti specifici per ogni combinazione.

Esempio pratico: gestione di un negozio (vendita e prestito)

1. Element e suoi figli

```
1 interface Elemento {
2     void accept(Visitor v);
3 }
4
5 class Libro implements Elemento {
6     String titolo;
7     Libro(String titolo) { this.titolo = titolo; }
8
9     public void accept(Visitor v) {
10         v.visitLibro(this);
11     }
12 }
13
14 class CD implements Elemento {
15     String titolo;
16     CD(String titolo) { this.titolo = titolo; }
17
18     public void accept(Visitor v) {
19         v.visitCD(this);
20     }
21 }
```

Listing 28: Element e ConcreteElement

2. Visitor e ConcreteVisitor

```
1 interface Visitor {
2     void visitLibro(Libro l);
3     void visitCD(CD c);
4 }
5
6 class VisitorVendita implements Visitor {
7     public void visitLibro(Libro l) {
8         System.out.println("Vendo il libro: " + l.titolo);
9     }
10
11     public void visitCD(CD c) {
12         System.out.println("Vendo il CD: " + c.titolo);
13     }
14 }
```

Listing 29: Visitor per la vendita

3. Aggiungiamo una nuova operazione: il prestito

```
1 class VisitorPrestito implements Visitor {
2     public void visitLibro(Libro l) {
3         System.out.println("Prestito il libro: " + l.titolo);
4     }
5
6     public void visitCD(CD c) {
7         System.out.println("Prestito il CD: " + c.titolo);
8     }
9 }
```

Listing 30: Visitor per il prestito

4. Uso della struttura con una lista

```
1 public class Negozio {
2     public static void main(String[] args) {
3         List<Elemento> articoli = List.of(
4             new Libro("Il nome della rosa"),
5             new CD("Greatest Hits")
6         );
7
8         Visitor vendita = new VisitorVendita();
9         Visitor prestito = new VisitorPrestito();
10
11         for (Elemento e : articoli) {
12             e.accept(vendita);
13         }
14
15         for (Elemento e : articoli) {
16             e.accept(prestito);
17         }
18     }
19 }
```

Listing 31: Negozio con lista di elementi

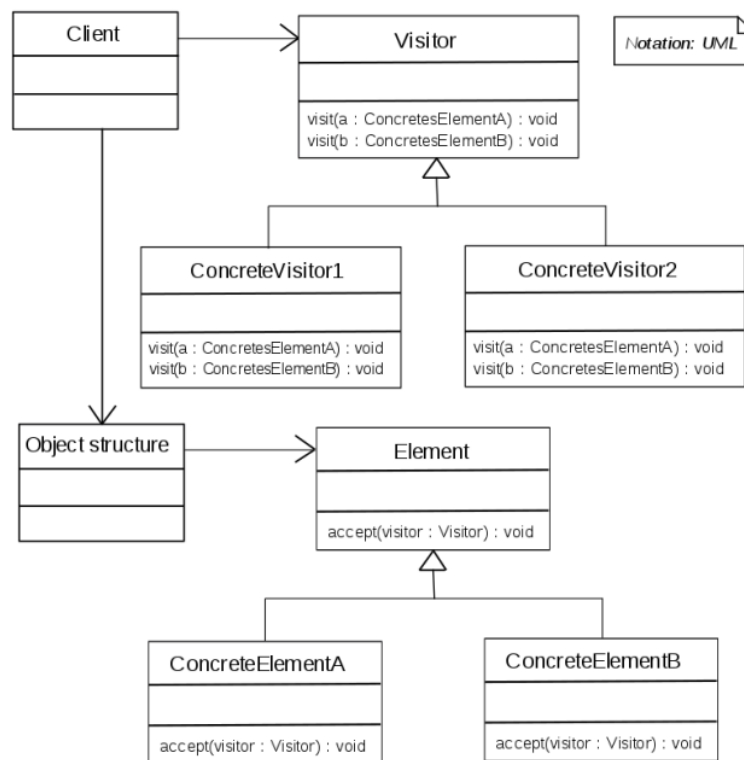
Conclusione

Il pattern Visitor è utile quando:

- la gerarchia di oggetti da visitare è stabile (non si aggiungono spesso nuove classi);
- le operazioni da eseguire sugli oggetti aumentano nel tempo.

Viceversa, se si aggiungono spesso nuovi tipi di oggetti, il Visitor diventa meno comodo perché bisogna modificare tutti i Visitor esistenti per supportare i nuovi tipi.

Diagramma UML:



Riepilogo Pattern

Pattern	Tipo	Scopo e struttura
1.17 – Singleton	Creazionale	Garantisce l'esistenza di una sola istanza di una classe, accessibile globalmente. Il costruttore è privato, e l'accesso avviene tramite un metodo statico <code>getInstance()</code> .
1.18 – Observer	Comportamentale	Un subject notifica automaticamente i suoi observer registrati quando cambia stato. Gli observer si registrano tramite <code>attach()</code> , e vengono aggiornati con <code>update()</code> quando il subject chiama <code>notify()</code> .
1.18.1 – Flyweight	Strutturale	Riduce l'uso di memoria condividendo oggetti simili. Divide lo stato in: <i>intrinseco</i> (condiviso nel flyweight) ed <i>estrinseco</i> (gestito dal client). Usato per oggetti numerosi, simili tra loro.

1.19 – Strategy	Comportamentale	Permette di incapsulare algoritmi intercambiabili sotto una comune interfaccia. Ogni strategia è una classe concreta, il client mantiene un riferimento astratto.
1.20 – Adapter	Strutturale	Consente di adattare l'interfaccia di una classe a quella attesa dal client. L'Adapter implementa l'interfaccia richiesta (Target) e converte le chiamate verso l'Adaptee.
1.21 – Decorator	Strutturale	Aggiunge dinamicamente funzionalità a un oggetto senza modificarne la struttura originale. Il Decorator implementa la stessa interfaccia del Component e mantiene un riferimento a esso.
1.22 – Composite	Strutturale	Permette di trattare oggetti singoli (foglie) e composti (nodi) in modo uniforme. Si basa su una gerarchia con Component , Composite , Leaf , con metodi comuni come add() e remove() .
1.23 – Visitor	Comportamentale	Permette di separare un algoritmo dalla struttura di oggetti su cui opera. Gli elementi accettano un visitor tramite accept(visitor) , che poi invoca visit() specifici. Supporta estensibilità (OCP) ma è fragile se cambiano le classi visitate.

1.24 – Modello LMU nei VCS con vantaggi e svantaggi

Nei **sistemi di controllo versione** (VCS), la gestione dell'accesso concorrente ai file è fondamentale. Uno dei modelli storicamente usati per evitare conflitti tra modifiche simultanee è il modello **Lock-Modify-Unlock** (LMU), impiegato da sistemi centralizzati come CVS o Subversion.

Il modello LMU prevede che:

- un utente, per modificare un file, debba prima eseguire un **lock** su di esso (blocco);
- una volta completata la modifica, debba eseguire **unlock** (sblocco), rendendo nuovamente il file disponibile agli altri utenti.

Svantaggi del modello LMU:

- Possibilità che un utente si dimentichi di sbloccare il file, impedendo ad altri di modificarlo e causando ritardi nello sviluppo.
- Serializzazione non necessaria: non tutte le modifiche concorrenti causano conflitti, ma il modello lo impedisce a prescindere.
- Falso senso di sicurezza: anche bloccando un file, possono verificarsi conflitti se altri file modificati da altri utenti dipendono da esso.
- Il modello richiede una connessione attiva alla rete, quindi **non è adatto al lavoro offline**.

Vantaggi del modello LMU:

- È adatto a gestire **file non unibili**, come immagini o binari, dove il merging automatico non è possibile.

1.25 - Modello CMM nei VCS con vantaggi e svantaggi

Il modello **Copy-Modify-Merge** (CMM) è utilizzato nei sistemi di controllo versione distribuiti (es. Git, Mercurial) e rappresenta un'evoluzione rispetto al modello Lock-Modify-Unlock. Questo approccio **non prevede meccanismi di blocco** dei file in fase di modifica.

Ogni sviluppatore esegue un **clone** della repository centrale, ottenendo una copia locale su cui lavorare in modo indipendente e parallelo rispetto agli altri. Le modifiche vengono successivamente unite (*merge*) nella repository centrale.

Fasi principali:

1. Ogni sviluppatore lavora su una copia personale del progetto.
2. Le modifiche vengono effettuate localmente.
3. Al momento del **push** o **merge**, le modifiche vengono integrate con quelle degli altri sviluppatori.

Esiti possibili del merge:

- **Successo:** nessuna modifica conflittuale, il merge avviene automaticamente.
- **Conflitto:** due o più sviluppatori modificano la stessa porzione di codice; occorre risolvere i conflitti manualmente.

Vantaggi:

- **Collaborazione simultanea:** più sviluppatori possono lavorare in parallelo senza blocchi.
- **Velocità:** si evitano i ritardi causati dall'attesa di sblocco dei file.
- **Flessibilità:** ogni sviluppatore lavora offline, con possibilità di eseguire commit locali.

Svantaggi:

- **Conflitti:** è necessario gestire i conflitti manualmente in caso di modifiche concorrenti.
- **Conflitti non rilevati:** il sistema non rileva conflitti concettuali o logici (es. cambiamenti compatibili sintatticamente ma che rompono la logica del programma).

Osservazione: Anche in presenza di conflitti, i tempi per la loro risoluzione sono generalmente inferiori rispetto ai ritardi introdotti dal sistema di lock previsto nel modello LMU.

Modulo 2

2.1 - Spiegare il modello a cascata e le sue criticità

Il **modello a cascata** (*waterfall model*) è un modello di processo per lo sviluppo software basato su una sequenza rigida di fasi, ciascuna delle quali deve essere completata interamente prima di passare alla successiva. Le fasi previste sono:

- **Studio di fattibilità**
- **Analisi dei requisiti**
- **Analisi del problema**
- **Progettazione**
- **Implementazione**
- **Collaudo**
- **Manutenzione**

Ogni fase produce dei *semilavorati* (es. documentazione tecnica, codice sorgente, sistema funzionante) utilizzati dalla fase successiva. Questo garantisce un certo controllo sulla qualità del lavoro svolto. Inoltre, il modello a cascata si basa anche sulla definizione anticipata di **date** e scadenze, utili per tracciare l'avanzamento del progetto.

Ipotesi implicite del modello:

- **Immutabilità dell'analisi:** si assume che il cliente sia in grado di definire sin dall'inizio tutte le funzionalità desiderate.
- **Immutabilità della progettazione:** si assume che il progetto possa essere definito completamente prima di iniziare a scrivere codice.

Vantaggi principali:

- Maggiore controllo e pianificazione del progetto
- Separazione chiara tra le fasi
- Produzione sistematica di documentazione

Criticità:

- **Rigidità:** è difficile tornare indietro per correggere errori o rivedere decisioni prese in fasi precedenti.
- **Evoluzione dei requisiti:** spesso i requisiti cambiano nel tempo, anche a causa della maggiore consapevolezza acquisita dal cliente.
- **Progettazione fragile:** un progetto interamente definito a priori può risultare inadeguato quando emergono nuove esigenze o vincoli tecnici.

Soluzioni alternative: Per ridurre questi limiti si è introdotta una versione modificata del modello, con **retroazione limitata**, che consente di tornare indietro di un solo livello in caso di necessità. Un altro approccio è il **throw-away prototyping**, in cui si realizza inizialmente un prototipo che viene successivamente scartato, prima di procedere alla costruzione del sistema reale seguendo il modello a cascata.

Osservazione: Anche se queste soluzioni mitigano alcuni svantaggi, rendono il processo più costoso, compromettendo i benefici economici del modello a cascata stesso.

2.2 - Spiegare il modello a cascata e il modello iterativo

Per una descrizione completa del **modello a cascata**, si rimanda alla domanda **2.1**.

Il **modello iterativo**, a differenza del modello a cascata, non segue una sequenza rigida di fasi ma si basa sull'idea di costruire il sistema attraverso una serie di **iterazioni incrementali**. Ogni iterazione produce un *prodotto parziale ma funzionante*, che viene poi revisionato, testato e migliorato nelle iterazioni successive.

Caratteristiche principali del modello iterativo:

- **Ciclicità:** il ciclo di vita del software è composto da passi ripetuti (analisi, progettazione, implementazione, test), ognuno dei quali porta a una versione migliorata del prodotto.
- **Incrementalità:** ad ogni iterazione vengono aggiunte funzionalità o migliorati i requisiti esistenti.
- **Feedback continuo:** grazie alla disponibilità di versioni funzionanti del software fin dalle prime iterazioni, è possibile raccogliere il feedback degli utenti e migliorare costantemente il prodotto.
- **Gestione del cambiamento:** il modello è particolarmente indicato in contesti dove i requisiti sono incerti o destinati a cambiare.

Vantaggi del modello iterativo:

- Migliore adattabilità ai cambiamenti nei requisiti
- Maggiore coinvolgimento del cliente e possibilità di ottenere feedback frequenti
- Riduzione dei rischi, grazie alla possibilità di individuare problemi nelle prime fasi del ciclo
- Possibilità di ottenere versioni funzionanti del software prima del completamento del progetto

Svantaggi:

- Può essere difficile stimare con precisione tempi e costi, poiché il numero di iterazioni può variare
- Richiede maggiore capacità di pianificazione e coordinamento del team
- Può generare confusione se non vengono stabiliti dei criteri chiari di completamento per ogni iterazione

Esempio di processo iterativo: Rational Unified Process (RUP) Uno dei processi di sviluppo che adotta un approccio iterativo è il **Rational Unified Process (RUP)**. Esso suddivide il progetto in 4 macrofasi (Inception, Elaboration, Construction, Transition), ognuna delle quali può contenere più iterazioni. In ogni iterazione si attraversano tutte le attività fondamentali (analisi, design, coding, testing), fino a ottenere una release stabile e migliorata del sistema.

Conclusione: Il modello iterativo si pone come una risposta flessibile ed efficace ai limiti del modello a cascata, soprattutto nei contesti moderni in cui l'evoluzione dei requisiti è inevitabile. Tuttavia, non è sempre adatto a progetti molto semplici o estremamente regolamentati, dove la prevedibilità assoluta è fondamentale.

2.3 - Rational Unified Process (RUP)

Il **Rational Unified Process (RUP)** è un modello di processo software **iterativo e incrementale** (si veda domanda **2.2**), pensato per progetti di grandi dimensioni. È considerato un modello **ibrido**, poiché integra elementi da tutti i principali modelli di sviluppo software.

Il RUP si articola secondo tre diverse **prospettive**:

1. Prospettiva dinamica: descrive l'evoluzione temporale del processo e si suddivide in quattro fasi principali:

1. Avvio (Inception):

- Definizione del *business case* e valutazione del contesto di mercato.
- Identificazione degli attori esterni e dei casi d'uso principali.
- Prima valutazione dei rischi.

2. Elaborazione (Elaboration):

- Definizione dell'architettura del sistema e analisi del dominio.
- Sviluppo di un'architettura eseguibile per i casi d'uso significativi.
- Requisiti per il completamento:
 - modello dei casi d'uso completo all'80%;
 - descrizione e sviluppo dell'architettura del sistema;

- revisione di business case e rischi;
- pianificazione del progetto.

- **Nota:** dopo questa fase, le modifiche all'architettura risultano costose e rischiose.

3. Costruzione (Construction):

- Implementazione parallela dei componenti.
- Test e integrazione delle componenti.
- Preparazione della documentazione.

4. Transizione (Transition):

- Rilascio del sistema all'utente finale.
- Formazione degli utenti.
- Esecuzione di beta testing per validare e verificare il prodotto.

2. Prospettiva statica: riguarda le **attività di produzione del software**, chiamate *workflow*. I principali sono:

1. **Modellazione delle attività aziendali:** analisi dei processi aziendali tramite business case.
2. **Requisiti:** identificazione degli attori e definizione dei casi d'uso.
3. **Analisi e progetto:** definizione e documentazione dell'architettura tramite modelli UML.
4. **Implementazione:** sviluppo dei componenti e generazione automatica del codice.
5. **Test:** verifica di sottocomponenti e sistema completo.
6. **Rilascio:** distribuzione del software agli utenti finali.

Sono inoltre presenti **3 workflow di supporto:**

- **Gestione della configurazione e delle modifiche:** tiene traccia delle evoluzioni del sistema.
- **Gestione del progetto:** pianifica e controlla l'avanzamento del lavoro.
- **Ambiente:** fornisce tool e strumenti di sviluppo al team.

3. Prospettiva pratica: suggerisce buone prassi da seguire nello sviluppo. Le sei principali sono:

1. **Sviluppare ciclicamente il software:** procedere per priorità.
2. **Gestire i requisiti:** documentare e analizzare tutte le richieste del cliente.
3. **Usare architetture basate sui componenti:** suddividere il sistema in unità riutilizzabili.

4. **Usare modelli visivi:** rappresentare staticamente e dinamicamente il sistema con UML.
5. **Verificare la qualità del software:** garantire il rispetto degli standard di qualità.
6. **Controllare le modifiche:** usare strumenti per la gestione del versionamento e delle modifiche.

2.4 – Tipologie di requisiti

I **requisiti** rappresentano la descrizione dei servizi offerti da un sistema software e dei vincoli entro cui deve operare. Si distinguono due tipologie principali:

- **Requisiti utente:**

- Descrivono i servizi offerti dal sistema e i vincoli generali.
- Sono formulati ad un livello **alto e astratto**, pensato per essere comprensibile anche a utenti non tecnici.
- Vengono spesso scritti in *linguaggio naturale*, accompagnati da *diagrammi* (es. casi d'uso).

Esempio: “Il sistema deve consentire all'utente di prenotare un biglietto per un treno tramite un'interfaccia web intuitiva.”

- **Requisiti di sistema:**

- Sono una specifica dettagliata di tutte le funzionalità e vincoli tecnici del sistema.
- Vengono utilizzati da progettisti, sviluppatori e tester.
- Possono essere suddivisi nelle seguenti categorie:

1. **Requisiti funzionali:**

- * Specificano *cosa deve fare* il sistema.
- * Descrivono il comportamento in risposta a determinati input o condizioni.

Esempio: “Se un utente inserisce credenziali errate per tre volte consecutive, il sistema deve bloccare l'accesso per 10 minuti.”

2. **Requisiti non funzionali:**

- * Definiscono le *caratteristiche qualitative* del sistema, come performance, sicurezza, usabilità.
- * Sono spesso vaghi, difficili da quantificare e possono entrare in conflitto tra loro o con i requisiti funzionali.

Sottocategorie principali:

- **Requisiti del prodotto:** vincoli sulle proprietà del software (es. tempo di risposta, affidabilità).
Esempio: “Il sistema deve rispondere a ogni richiesta in meno di 2 secondi.”
- **Requisiti organizzativi:** vincoli sul processo di sviluppo (es. standard di codifica, linguaggi usati).
Esempio: “Il software deve essere sviluppato in Java secondo lo standard aziendale XYZ.”
- **Requisiti esterni:** derivano da vincoli normativi, etici o ambientali.
Esempio: “Il sistema deve rispettare il GDPR in tutte le operazioni che coinvolgono dati personali.”

3. **Requisiti del dominio:**

- * Sono specifici del *dominio applicativo* del sistema (es. bancario, sanitario, industriale).
- * Possono implicare conoscenze tecniche di settore e sono generalmente forniti dagli *esperti del dominio*.

Esempio: “Nel sistema bancario, ogni bonifico deve essere autorizzato tramite token OTP entro 30 secondi dalla generazione.”

2.5 – Ciclo di vita della valutazione del rischio

L'**analisi del rischio** ha lo scopo di bilanciare le potenziali perdite causate da attacchi informatici con i costi necessari per garantire la protezione dei beni. Una sua componente fondamentale è la **valutazione del rischio**, articolata in diverse fasi.

- **Valutazione preliminare del rischio:** consente di determinare i *requisiti di sicurezza* dell'intero sistema. Si svolge nelle prime fasi dello sviluppo.
- **Ciclo di vita della valutazione del rischio:** accompagna lo sviluppo del sistema durante tutto il ciclo di vita del software. Questa fase si basa su:
 - la conoscenza dell'**architettura del sistema** e dell'**organizzazione dei dati**;
 - le scelte già effettuate riguardo a *piattaforme, middleware e strategia di sviluppo*;
 - l'individuazione delle **vulnerabilità** potenziali, cioè quali beni sono maggiormente esposti a minacce;
 - la valutazione della **probabilità di attacco** e delle possibili **conseguenze**.
- Il risultato finale di questa analisi consiste in un insieme di **decisioni ingegneristiche** che influenzano:
 - la progettazione tecnica del sistema;
 - le misure di protezione adottate;
 - eventuali limiti d'uso imposti al sistema stesso.

2.6 – Principali categorie di requisiti per la sicurezza

Lo scopo dei **requisiti di sicurezza** è definire i comportamenti considerati inaccettabili per il sistema, senza specificare direttamente le funzionalità da implementare.

Essi indicano:

- il **contesto** operativo;
- i **beni da proteggere**;
- il **valore** che tali beni rivestono per l'organizzazione.

Le principali **categorie** di requisiti di sicurezza sono:

- **Requisiti di identificazione:** indicano se il sistema deve identificare l'utente prima di qualsiasi interazione.
- **Requisiti di autenticazione:** stabiliscono i meccanismi per verificare l'identità dell'utente (es. password, smartcard, biometria).
- **Requisiti di autorizzazione:** definiscono i permessi e privilegi che gli utenti possono esercitare una volta identificati e autenticati.

- **Requisiti di immunità:** stabiliscono le difese del sistema contro malware e software malevolo.
- **Requisiti di integrità:** specificano i meccanismi per evitare la corruzione o modifica non autorizzata dei dati.
- **Requisiti di scoperta delle intrusioni:** indicano le tecniche e strumenti adottati per rilevare accessi non autorizzati o attacchi.
- **Requisiti di non-ripudio:** garantiscono che un attore coinvolto in una transazione non possa negare la propria partecipazione.
- **Requisiti di riservatezza:** stabiliscono come proteggere le informazioni da accessi non autorizzati (es. crittografia).
- **Requisiti di controllo della protezione:** definiscono come monitorare e verificare l'uso delle risorse protette.
- **Requisiti di protezione della manutenzione:** assicurano che l'attività di manutenzione non comprometta involontariamente la sicurezza del sistema.

2.7 – Linee guida di progettazione nella sicurezza

Le principali linee guida da seguire nella progettazione di un sistema sicuro sono:

1. **Basarsi su una politica esplicita per le decisioni inerenti la sicurezza.**
Le politiche di sicurezza sono documenti di alto livello che definiscono i criteri di sicurezza (chi può accedere a cosa e in quali condizioni), senza specificare i meccanismi concreti per raggiungerli. È importante incorporare tali politiche nella progettazione.
2. **Evitare ogni singolo punto di fallimento.**
Il fallimento di un singolo componente non deve compromettere l'intero sistema.
3. **Fallire in modo controllato.**
Il sistema deve degradare in modo sicuro, evitando comportamenti erratici o pericolosi in caso di guasti.
4. **Bilanciare sicurezza e usabilità.**
Maggiore sicurezza spesso comporta minore facilità d'uso. Occorre trovare un compromesso per evitare che gli utenti aggirino le misure di sicurezza.
5. **Essere consapevoli dei rischi dell'ingegneria sociale.**
L'ingegneria sociale può indurre gli utenti a rivelare dati sensibili: il sistema deve limitarne gli effetti con processi e formazione.
6. **Utilizzare ridondanza e diversità per ridurre i rischi.**
 - *Ridondanza:* mantenere più copie di dati o versioni del software.

- *Diversità*: utilizzare tecnologie differenti per evitare punti di fallimento comuni dovuti a vulnerabilità condivise.

7. Validare l'input.

Tutti i dati di ingresso devono essere controllati per prevenire attacchi come *SQL Injection*, *Buffer Overflow*, ecc.

8. Organizzare le informazioni in compartimenti.

Gli utenti devono poter accedere solo ai dati di loro competenza, limitando la visibilità e l'esposizione del sistema.

9. Progettare per il deployment.

Includere nel sistema strumenti per facilitare il deployment e la configurazione sicura. Alcune buone pratiche:

- supporto per la visualizzazione e analisi delle configurazioni;
- privilegi di default minimi;
- configurazioni locali;
- meccanismi per correggere vulnerabilità.

10. Progettare per il ripristino.

Prevedere meccanismi automatici o guidati per aggiornare e correggere il sistema in caso di attacchi o vulnerabilità.

2.8 – White Box e Black Box Testing

Esistono due principali approcci per il testing dei sistemi software:

• Black Box Testing

In questo approccio il sistema viene testato senza conoscere il codice sorgente né l'implementazione interna. Il focus è sul comportamento esterno del sistema. I tester osservano gli input e verificano gli output, cercando di dedurre la struttura interna attraverso l'interazione. Questo metodo è utile per:

- Valutare l'affidabilità e la velocità del sistema;
- Simulare il punto di vista dell'utente esterno o di un potenziale attaccante;
- Scoprire vulnerabilità legate a configurazioni errate, errori nei protocolli, o limiti nei linguaggi di programmazione utilizzati.

• White Box Testing

In questo approccio i tester hanno pieno accesso alle informazioni interne del sistema: configurazioni di rete, impostazioni delle macchine virtuali, e soprattutto il codice sorgente. Questo consente:

- L'analisi approfondita della logica interna;
- La revisione del codice (code review) per verificarne leggibilità e modularità;

- La creazione di test ad hoc basati sulle parti più deboli del sistema.

Entrambi i metodi sono complementari: il Black Box è efficace per testare il sistema come un utente o attaccante, mentre il White Box è adatto a test dettagliati e ottimizzazione interna.

2.9 – Capacità di sopravvivenza del sistema

La **capacità di sopravvivenza del sistema** descrive l'abilità di un sistema di continuare a fornire servizi agli utenti autorizzati anche in caso di attacchi informatici o componenti danneggiate.

Questa proprietà riguarda l'intero sistema e non le singole componenti, ed è fondamentale poiché economia e società sono sempre più dipendenti dai servizi digitali.

Obiettivi della sopravvivenza

Nel progettare sistemi resilienti è necessario:

- Identificare i **servizi critici**;
- Comprendere le **modalità di attacco** verso tali servizi;
- Stabilire lo **standard minimo di qualità** da garantire;
- Definire **strategie di protezione** durante un attacco;
- Progettare il **ripristino** rapido del sistema.

Survivable Analysis System (SAS)

Per affrontare in modo strutturato la sopravvivenza di un sistema si utilizza il **Survivable Analysis System**, un metodo basato su 3 strategie e 4 fasi.

Strategie

- **Identificazione:** individuazione tempestiva di attacchi o malfunzionamenti e valutazione del danno;
- **Resistenza:** capacità di contenere e resistere agli attacchi;
- **Ripristino:** mantenimento delle funzionalità essenziali e ripristino completo dopo l'attacco.

Fasi del processo

1. **Comprendere il sistema:** analizzare architettura, requisiti e obiettivi;
2. **Identificare i servizi critici:** capire quali servizi e componenti sono fondamentali;
3. **Simulare attacchi:** analizzare scenari realistici e punti deboli del sistema;
4. **Analizzare la sopravvivenza:** identificare componenti vulnerabili ed essenziali, valutando le strategie di identificazione, resistenza e ripristino.

Criticità dell'approccio

Nonostante la sua importanza, l'analisi della sopravvivenza è spesso trascurata, soprattutto da aziende che non hanno ancora subito attacchi. Tuttavia, investire in prevenzione e resilienza è fondamentale per evitare danni ingenti in futuro, sia in termini economici che di sicurezza personale.